An Auditable Confidentiality Protocol for Blockchain Transactions

Aoxuan Li[a,b], Gabriele D'Angelo[d], Su-Kit Tang[c], Frank Fang[e], Baron Gong[e]

[a]*IMDEA Software Institute, Madrid, Spain*
[b]*Universidad Politécnica de Madrid, Madrid, Spain*
[c]*Faculty of Applied Sciences, Macao Polytechnic University, Macao SAR, China*
[d]*Department of Computer Science and Engineering, University of Bologna, Bologna, Italy*
[e]*Expand ZK, Singapore*

Abstract

Blockchain technology inherently exposes user payment details, including account balances, asset holdings, and transaction histories, posing significant security and privacy risks. Moreover, no existing cross-chain bridges support private transactions that protect users' trading histories. This paper introduces a novel confidentiality protocol that conceals user payments while ensuring auditability. The protocol employs a privacy committee, where encrypted secret shares of transaction data are distributed among committee members. Auditors can access private transaction data only with majority approval from the committee. Additionally, the protocol utilizes a ZK-rollup scheme, batching multiple transactions to reduce private transaction costs by 90% compared to non-batched approaches. Implemented using Zokrates and Solidity, the proposed scheme has been evaluated on the Ethereum test network, demonstrating a one-to-one private transaction latency of approximately 5 seconds. The protocol's security is rigorously analyzed using the standard real/ideal world paradigm.

Keywords: Privacy, Blockchains, Cryptocurrency, Smart contracts, Zero-knowledge proofs, Auditability

1. Introduction

Blockchain technology inherently exposes user transaction data, including account balances, asset holdings, and trade activities, due to its transparency and traceability features. This data exposure leads to significant security and privacy concerns, hindering the broader adoption of blockchain technology. The blockchain ecosystem's reliance on interoperability across various blockchains and applications has led to the formation of numerous cross-chain bridges. These bridges facilitate asset transfers between chains, accumulating billions in inter-chain trading volumes. However, no existing cross-chain bridge supports private transactions that protect users' trading histories.

Several privacy-preserving transaction protocols have been proposed, such as Zcash [1, 2], Monero [3], and Tornado Cash. These systems employ cryptographic techniques like zero-knowledge proofs, ring signatures, and coin mixing to conceal transaction details. However, they suffer from limitations such as lack of interoperability across blockchains, high transaction fees, or the inability to provide auditability.

Existing single-chain private transaction solutions, such as coin mixing systems, are not user-friendly. These systems impose high transaction fees and complicate asset transfers between chains. For instance, conducting private cross-chain transactions requires using single-chain coin mixers on

supported chains before completing the cross-chain operation. Additionally, private transactions often incur high gas fees, further burdening users. For example, there have been attempts to integrate existing privacy-preserving solutions with other networks. RedBridge [4] enables the transfer of Zcash [2] to the Avalanche blockchain. However, such integrations remain limited in scope compared to bridges supporting widely adopted blockchains like Bitcoin and Ethereum. Additionally, these solutions still inherit the limitations of single-chain privacy mechanisms, such as high transaction costs and complex workflows.

On August 8, 2022, the Office of Foreign Assets Control (OFAC) of the U.S. Department of the Treasury sanctioned Tornado Cash, citing it as "a substantial danger to national security". Regulators are concerned about the lack of visibility into suspicious transaction flows and the potential for money laundering. They are unable to differentiate between the assets of criminal actors and those of legitimate users who seek privacy. Nevertheless, many legitimate users require private transactions to protect their trading histories and maintain a competitive advantage.

Blockchain transparency allows anyone to track transactions, which poses a significant challenge for users who require financial privacy. Consider a corporation making payroll payments on a public blockchain. If employee salaries are directly transferred, competitors and external observers can infer sensitive financial details about the company's compensation structure. A privacy-preserving mechanism would enable confidential payroll transactions while allowing auditors to verify compliance with tax regulations.

Similarly, decentralized finance (DeFi) protocols often require large institutional trades, where exposing trade history can lead to front-running attacks. Traders executing large swaps on automated market makers (AMMs) risk price slippage if their transactions are publicly visible before settlement. A confidentiality protocol can mitigate this issue by concealing trade details until finalization while ensuring that auditors retain the ability to review transactions for compliance purposes.

Our protocol ensures both *confidentiality* and *auditability* in cross-chain transactions. Unlike Zcash, which operates as a standalone blockchain and lacks native interoperability with Ethereum, Binance Smart Chain, or Polygon, our approach allows privacy-preserving transactions across different chains while enabling regulated access when necessary. Additionally, while systems like Tornado Cash provide anonymity, they lack built-in mechanisms for accountability, raising concerns about illicit usage. Our protocol addresses this gap by introducing an auditable privacy framework where authorized auditors can verify transactions under a threshold consensus mechanism.

This paper addresses these issues by proposing an auditable confidentiality protocol for blockchain transactions. Our protocol ensures user privacy while allowing necessary regulatory auditing. It supports popular blockchains and cross-chain transactions, employing a ZK-rollup scheme for scalability and cost efficiency. Additionally, a decentralized auditing system provides transparency without compromising user privacy.

Our contributions are as follows:

- Designed a **confidential** protocol for blockchain transactions supporting popular chains.
- Implemented a versatile protocol for both **single-chain** and **cross-chain** transactions, allowing the sender and receiver to be on the same or different chains.
- Introduced the **JoinSplit** mechanism, enabling users to execute internal transfers seamlessly.
- Employed the **ZK-rollups** scheme to enhance protocol throughput and scalability. Zero-Knowledge Rollups bundle multiple transactions off-chain and submit a single proof on-chain,

reducing computational load and transaction fees while maintaining transaction validity through zero-knowledge proofs.

- Ensured protocol **confidentiality** while incorporating **auditability** for auditors. The protocol does not disclose users' transaction data unless a significant consensus among auditors is reached.

1.1. Paper organization

This paper is organized as follows: Section 2 provides the necessary background on blockchain, zero-knowledge proofs, and summarizes previous work. Section 3 offers an overview of the proposed solution. Section 4 defines the protocol, and Section 5 describes its construction. We formally prove completeness and security in Section 6. Section 7 presents benchmarks for our protocol implementation. Finally, Section 8 summarizes our contributions and discusses future work.

2. Background

2.1. Blockchain

A blockchain is a distributed ledger shared among a network of computers. The first generation of blockchain technology is epitomized by Bitcoin [5], which uses a blockchain to record peer-to-peer transactions. Ethereum [6] introduced *smart contracts*, self-executing programs on the blockchain that operate when predefined conditions are met, enabling decentralized applications. As the blockchain grows, all nodes in the network must agree on the current state and append the same block. To ensure consistency among nodes, blockchains utilize a set of protocols known as *consensus algorithms*.

2.2. Zero-Knowledge Proof

In 1985, Goldwasser, Micali, and Rackoff [7] introduced the concept of zero-knowledge proof, allowing a prover to demonstrate the truth of a statement without revealing any information beyond the validity of the statement. For instance, a prover can prove that two graphs are isomorphic without disclosing the isomorphism itself. Blum, Feldman, and Micali [8] later introduced non-interactive zero-knowledge proof, where the proof is publicly verifiable without needing interaction with the prover. Early zero-knowledge proof protocols were often impractical due to inefficiency and large proof sizes. The zk-SNARK (Zero-Knowledge Succinct Non-Interactive Argument of Knowledge) algorithm [9, 10] is one of the first practical protocols, providing succinct proofs independent of the statement's complexity. This paper employs the Groth16 [11, 12] construction of zk-SNARK, known for its efficiency and compact proof size.

2.3. Previous Works

Zerocash/Zcash[1, 2]: Pioneering the realm of privacy-preserving blockchains, Zerocash stands as the inaugural implementation utilizing zero-knowledge proofs. Building upon this foundation, subsequent advancements include Zexe [13], Zkay [14], and ZeeStar [15]. These works extend the privacy-preserving framework to arbitrary smart contracts through the application of the Groth16 construction.

While Zexe mandates users to compute smart contracts off-chain and subsequently submit zero-knowledge proofs for correctness, it lacks dedicated development tools. In contrast, Zkay and its subsequent iteration, Zeestar, introduce a language tailored for privacy-preserving smart

contracts. However, these protocols exhibit significant computational intensity, demanding substantial resources such as multiple CPU cores (Zkay, 12 cores) or extensive memory usage (Zexe, 256 GB of RAM).

Monero [3]: Diverging from Zerocash, Monero, a privacy-preserving blockchain rooted in the Bitcoin protocol, relies on ring signatures and range-proofs [16]. Range-proofs, a variant of zero-knowledge proofs, establish the validity of a value within a defined range. For instance, Monero users must verify that inputs and outputs in their transactions fall within a valid range to prevent overflow. Zether [17], akin to Monero but built upon Ethereum, encounters inherent scalability limitations due to the variable size of range-proofs. Other works inspired by Monero, such as MatTiCT(+) [18, 19] and RingCT2.0/3.0 [20, 21], face challenges such as non-constant zero-knowledge proof size and verification time. Additionally, these Monero-like solutions offer privacy within small anonymity sets.

Cross-Chain Solutions: Traditional privacy-preserving blockchain solutions, like Zerocash and Monero, typically confine privacy to single-chain transactions. Zerocross [22] innovates by providing a privacy-preserving cross-chain solution for Monero. Utilizing a sidechain and zero-knowledge proofs for set membership [23], Zerocross facilitates cross-chain transactions. However, it should be noted that this solution only supports cross-chain transactions with Monero, with the caveat that sender and receiver identities remain visible to each other. P2DEX [24], a privacy-preserving exchange utilizing publicly verifiable secure multiparty computation (MPC) [25], faces limitations in supporting high-frequency trading due to the inherent expense of MPC. Other approaches, such as those relying on special hardware like Trusted Execution Environments (TEEs) [26, 27], prove impractical and may pose serious vulnerabilities [28, 29].

Auditability Solutions: Many early privacy-preserving blockchain solutions lack robust auditability, making them susceptible to illicit activities. ZkLedger [30] and Fabzk [31] address this limitation by utilizing homomorphic commitments and non-interactive zero-knowledge proofs (ZKPs) to enable auditing. However, their designs focus on cross-organization transactions rather than broader blockchain ecosystems. ZEBRA [32] integrates auditability within an anonymous credential framework, ensuring compliance while maintaining privacy. Similarly, Azeroth [33] and ACA [34] introduce privacy-preserving transactions with auditing capabilities, though they lack support for cross-chain interactions. More recently, the *Privacy Pools* concept [35] employs ZKPs and smart contracts to allow users to prove regulatory compliance without revealing their complete transaction history. While effective in balancing privacy and oversight, it does not fully address scalability constraints or extend support to cross-chain environments. Our proposed auditable confidentiality protocol advances these efforts by integrating a **privacy committee with encrypted secret shares**, ensuring that auditors can access private transaction data only through majority approval. This mechanism enhances compliance while preserving user privacy, making auditability more secure and adaptable to different blockchain settings.

Scalability Solutions: Scalability remains a critical challenge for existing auditable privacy solutions. Systems like ZkLedger [30] and Fabzk [31] experience performance degradation as user numbers grow, limiting their feasibility for high-throughput blockchain networks. Similarly, Privacy Pools rely on on-chain proof verification, which can introduce significant computational overhead. To address these limitations, our protocol leverages **ZK-rollups**, which aggregate multiple transactions into a single proof, drastically reducing computational and storage costs. Unlike previous solutions [36, 37, 38], it supports both **single-chain and cross-chain transactions**, enabling seamless interoperability between different blockchain ecosystems. By optimizing scalability while maintaining strong privacy and auditability guarantees, our approach provides a more efficient and adaptable solution for real-world blockchain applications.

The comparative analysis of these works is presented in Table 1.

Protocol	**Privacy**	**Auditability**	**Scalability**	**Cross-Chain Support**
Zerocash/Zcash	Yes	No	Low	No
Zexe	Yes	No	Low	No
Zkay	Yes	No	Low	No
Zeestar	Yes	No	Low	No
Monero	Yes*	No	Low	No
Zether	Yes*	No	Low	No
MatTiCT(+)	Yes*	No	Low	No
RingCT2.0/3.0	Yes*	No	Low	No
Zerocross	Yes*	No	Low	Yes**
P2DEX	Yes	No	Low	Yes
Tesseract/TEEs	Yes	No	Low	No
ZkLedger	Yes	Yes	Low	No
Fabzk	Yes	Yes	Low	No
ZEBRA	Yes	Yes	Low	No
Azeroth	Yes	Yes	Low	No
ACA	Yes	Yes	Low	No
Privacy Pools	Yes	Yes	Low	No
Our Protocol	**Yes**	**Yes**	**High**	**Yes**

Table 1: Comparison of confidential protocol for Blockchain Transactions. ∗Protocols [3, 17, 18, 19, 20, 21, 22] provide privacy only within small anonymity sets. ∗∗ Monero only

3. Overview

In a cross-chain or single-chain transaction, the source blockchain network transfers coins to the destination blockchain network. In such protocols, the sender and receiver addresses are in plain text, making it possible to track the transaction graph. Many research studies indicate that this setup cannot ensure privacy [16, 39].

To address this issue, we employ a solution similar to Zcash [1, 2], where transactions are encrypted with the receiver's public key. The receiver can then locate the transaction and withdraw the coin. However, Zcash, a fork of Bitcoin [5], operates as a layer-1 blockchain and lacks integration with other blockchain networks such as Ethereum [6], BSC [40], and Polygon [41]. Consequently, Zcash does not support single-chain or cross-chain transactions across these popular chains. Additionally, Zcash transaction sizes can be ten times larger than Bitcoin, impacting scalability. While encrypted transactions provide privacy, they can be misused for criminal activities. Zcash's viewing keys allow external viewers to track transactions, but this scheme only reveals incoming transactions. Auditors cannot view a sender's address or outgoing transactions. Therefore, a more comprehensive audit feature is needed, where all transactions are auditable.

In this paper, we propose a protocol where the sender from one blockchain conceals transaction details using zero-knowledge proofs, allowing only the designated receiver to unveil the secret on another blockchain.

Suppose a user u on *Blockchain A* wants to send a coin valued v to u_1 on *Blockchain B*, where v belongs to some predefined values $\mathbb{V}$. Let $PRF_x^{addr}(\cdot)$, $PRF_x^{serial}(\cdot)$, and $PRF_x^{r}(\cdot)$ be pseudo-random functions that output the address, serial number, and random value r, respectively. Our protocol operates in two phases: *deposit* and *withdraw*.

During the deposit phase, user u on *Blockchain A* generates an address $addr := PRF_x^{addr}(u)$ and deposits the coin valued v with this address on *Blockchain B*. The transaction details, including the amount v, are encrypted and stored in a commitment scheme on *Blockchain B*.

During the withdrawal phase, user u_1 on *Blockchain B* generates a serial number $serial := PRF_x^{serial}(u_1)$ and a random value $r := PRF_x^{r}(u_1)$, then uses these values to create a zero-knowledge proof that verifies the transaction's validity without revealing any details. The proof is submitted to *Blockchain B*, where it is verified, allowing u_1 to withdraw the coin.

The implementation, named **Mystiko**, operates in two phases: **Mystiko Deposit** and **Mystiko Withdraw**. During the **Mystiko Deposit** phase, a user sends coins from a source chain to a destination chain via a bridge, and Mystiko locks those coins on the source chain. Notably, Mystiko uses the bridge as a data bridge instead of an asset bridge; i.e., the cross-chain bridge transfers smart contract events, not the actual tokens/cryptocurrency. Moreover, all private notes are encrypted, and only the user with the corresponding private key can decrypt them, generate a valid zero-knowledge proof, and withdraw the coin.

If the receiver wants to withdraw the coins, they generate a withdrawal transaction off-chain and verify it on-chain. Mystiko maintains a Merkle tree for all deposited coins and updates it when new coins are added. This operation could be expensive if executed on-chain due to high gas costs, computational complexity, and the need for frequent state updates, especially as the number of coins grows. Mystiko addresses this issue using **ZK-rollups**. Specifically, a rollup miner pulls on-chain deposits locally, calculates a Merkle tree root, generates a zero-knowledge proof that the Merkle tree root is correct, and validates it. The user then sends the proof with the root to the contract, and if the proof is validated, the Merkle tree root is updated.

We provide a detailed explanation of the protocol's architecture, as illustrated in Figure 1. The figure represents the flow and key components involved in enabling private and auditable cross-chain blockchain transactions.

1. Mystiko Deposit Phase:

- **User Initiation:** The process begins when a user on the source chain initiates a deposit. The user deposits tokens along with a rollup fee to a cross-chain bridge.
- **Data Handling:** The deposit transaction includes zk-SNARK parameters and a commitment. Mystiko locks the tokens on the source chain and saves the transaction details.
- **Cross-Chain Synchronization:** The bridge network synchronizes the state between the source and destination chains, transferring the necessary transaction data.
- **Lock-Mint Mechanism**. Our protocol employs a lock-mint model for cross-chain transfers. When a user deposits coins on the source chain, they are locked in a contract, and an equivalent amount of wrapped tokens is minted on the destination chain.

2. Mystiko Withdraw Phase:

- **User Request:** The user on the destination chain generates a withdrawal request. This request includes a zero-knowledge proof and the recipient's address.

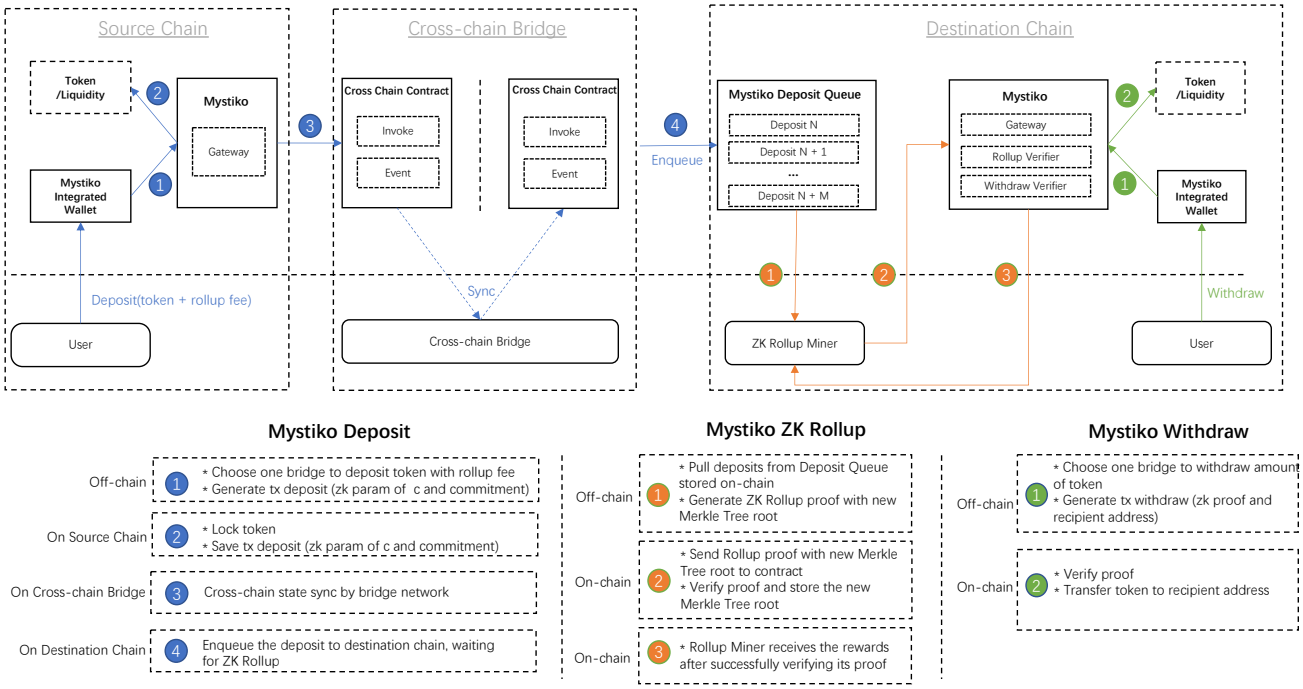

Figure 1: Architecture overview of the implementation. Transactions from the source chain are synced to the destination chain via a cross-chain bridge. The system enables **deposit**, **withdraw**, and **ZK-rollup** through a set of smart contracts.

- **Proof Verification:** The smart contract on the destination chain verifies the zero-knowledge proof. Upon successful verification, the tokens are transferred to the recipient's address.

3. Mystiko ZK-Rollup:

- **Rollup Miner Role:** A rollup miner collects on-chain deposits locally, calculates a new Merkle tree root, and generates a zero-knowledge proof that this root is correct.
- **Efficiency and Cost Reduction:** The rollup miner submits the proof and the new Merkle tree root to the contract. This process updates the Merkle tree root, reducing computational load and transaction fees.
- **Incentives:** The rollup miner receives rewards upon successfully verifying the proof.

Components:

- **Source Chain and Destination Chain:** Represent the two different blockchain networks between which the transaction occurs.
- **Cross-Chain Bridge:** Facilitates the synchronization and transfer of data between the source and destination chains without transferring actual tokens.
- **Smart Contracts:** Deployed on both chains to manage the deposit, withdraw, and rollup transactions.
- **Merkle Tree:** Used to maintain the state of all deposits, ensuring data integrity and enabling efficient proof verification.

Process Flow:

1. The user deposits tokens and rollup fees on the source chain.
2. The transaction details are synchronized across the bridge network.
3. The rollup miner processes the deposits, generates proofs, and updates the Merkle tree root on the destination chain.
4. The user requests a withdrawal on the destination chain, where the proof is verified, and tokens are released.

This architecture ensures that transactions remain private and auditable, leveraging zk-SNARKs and ZK-rollups to enhance scalability, reduce costs, and provide robust cross-chain support. The use of a privacy committee and encrypted secret shares further ensures that auditability is maintained while preserving user privacy.

4. Description of the Protocol

In this section, we provide an overview of the crucial data structures and algorithms integral to the protocol's operation. These include fundamental elements such as ledgers, public parameters, addresses, coins, transactions, and auditing processes. Table 2 provides an overview of the symbols used in the cryptographic protocol; further details will be provided later.

Our protocol introduces three key entities: **Users**, **Rollup miners**, and **Auditors**, each playing a crucial role in maintaining confidentiality and auditability in blockchain transactions.

- **Users**: Users initiate transactions by depositing funds on a source blockchain and withdrawing them on a destination blockchain. They interact with the system using cryptographic commitments and zero-knowledge proofs to ensure transaction privacy.
- **Rollup miners**: Rollup miners aggregate multiple transactions into a single ZK-rollup proof, reducing on-chain computation costs and improving scalability. They play a key role in updating the Merkle tree state across blockchains.
- **Auditors**: Auditors in real-world scenarios typically include law enforcement agencies, accounting firms, or regulatory bodies. Since there are no universally established rules for auditing private blockchain transactions, our protocol is designed to be flexible regarding the number of committee members and the threshold required for consensus. The protocol allows system designers to configure the number of auditors and define the quorum necessary to access transaction details. This ensures adaptability across different regulatory environments.

  Membership in the auditor committee is also flexible. The system can integrate governance-based selection, where stakeholders vote for committee members, or use decentralized identity verification to ensure the integrity of auditors. Future implementations may allow regulatory authorities to appoint auditors based on jurisdictional requirements.

The interactions between the entities follow a structured flow. In the *deposit phase*, a user deposits coins on the source blockchain, and the transaction details are encrypted and recorded as a commitment. During the *rollup phase*, rollup miners batch multiple deposit transactions into a ZK-rollup proof and update the Merkle tree on the destination blockchain. In the *withdrawal phase*,

the receiving user submits a zero-knowledge proof to claim the deposited funds, ensuring that only the rightful owner can withdraw. Finally, the *auditability mechanism* allows auditors to decrypt and verify transaction commitments if a majority consensus is reached, ensuring transparency without compromising user privacy.

Figure 2 illustrates the user deposit, zk-rollup, and withdrawal process, showing how transactions flow between the source and destination ledgers, with coins being deposited and withdrawn through the zk-rollup mechanism. Additionally, we explore the Merkle tree update process in Figure 3, which demonstrates the role of the zk-rollup transaction in updating the commitment tree. The Figure 4 also shows the role of auditors in jointly decrypting messages and recovering commitments.

Symbol	**Description**
L^{src}	Source chain's ledger
L^{dst}	Destination chain's ledger
$L_T^{\{src,dst\}}$	Ledger accessible at time T for both source and destination chains
pp	Public parameters list available to all users
$addr_{pk}$	Address public key
$addr_{sk}$	Address secret key
nk	Nullifier key used for generating serial numbers of receiving coins
pk_u^a	User's audit public key
sk_u^a	User's audit secret key
$pk_\cdot^a$	Auditor's audit public key
$sk_\cdot^a$	Auditor's audit secret key
$cm(c)$	Commitment of coin c
$v(c)$	Denomination of coin c
$sn(c)$	Serial number of coin c
$addr_{pk}(c)$	Address public key of coin c
$tx_{deposit}$	Deposit transaction, a tuple $(cm, v, *)$
$tx_{withdraw}$	Withdraw transaction, a tuple $[(rt, sn)]$,cm_1^{new}, cm_2^{new},v^{pub}, $ADDR$, π_{WITHDRAW}, $[msg_1^a, msg_2^a, \dots]$
tx_{rollup}	ZK-rollup transaction, a tuple $(rt^{old}, rt^{new}, hash_{[cm]}, pathIndices, N^{rollup}, \pi_{\mathsf{ROLLUP}}, *)$
$CMList_T$	List of all commitments in deposit transactions in L_T^{src}
$SNList_T$	List of all serial numbers in withdraw transactions in L_T^{src}
$Tree_T$	Merkle tree over $CMList_T$ at time T
rt_T	Root of the Merkle tree $Tree_T$ at time T
$Path_T(cm)$	Authentication path function for coin commitment cm
Q_T^{cm}	Queue of commitments waiting for rollup at time T
Π	Protocol, a tuple of algorithms (Setup, CreateAddress, CreateAuditKey, Deposit, Withdraw, Rollup, VerifyTransaction, Receive, Audit)
λ	Security parameter for the system
sk_u^a	User's secret key in the audit process

Symbol	Description
pk_u^a	User's public key in the audit process
$[msg_1^a, msg_2^a, \ldots]$	Encrypted messages for auditing
N^{rollup}	Number of commitments being rolled up
$pathIndices$	Direction selector for authentication path in ZK-rollup
rt^{old}	Old Merkle tree root
rt^{new}	New Merkle tree root
$hash_{[cm]}$	Hash of coin commitments
cm	Coin commitment
cm_1^{new}, cm_2^{new}	New coin commitments
π_{ROLLUP}	Proof for ZK-rollup

Table 2: List of Symbols and Their Descriptions

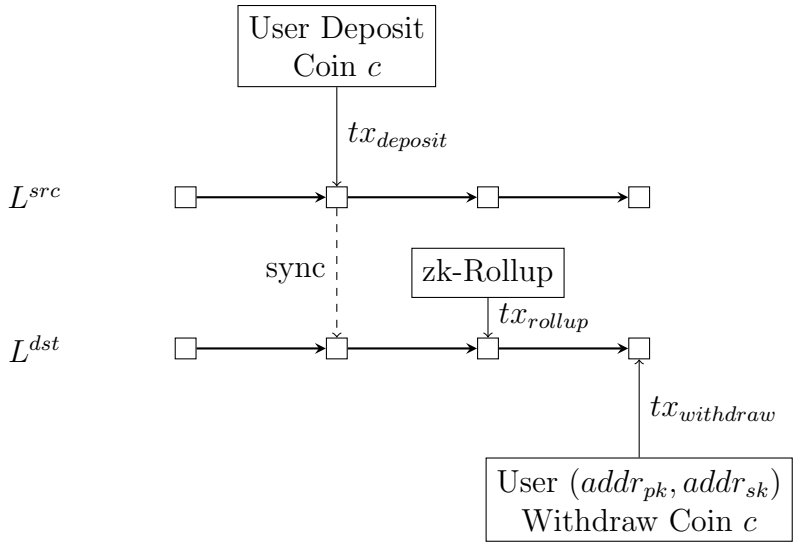

Figure 2: User deposit, zk-Rollup, and withdrawal process with transactions.

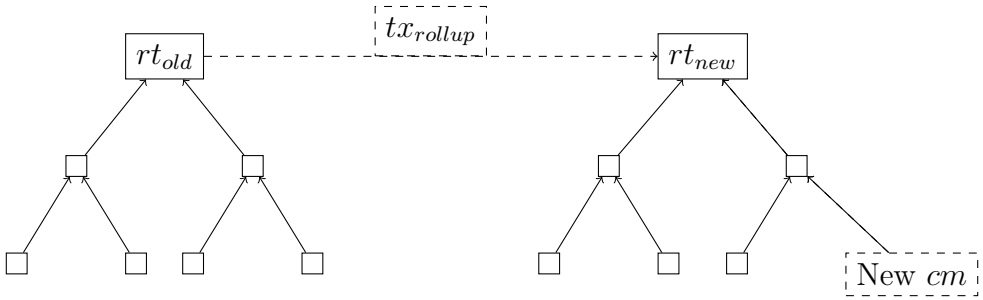

Figure 3: Merkle Tree Update Process with tx_{rollup}.

4.1. Data Structures

The main data structures used in the protocol are described in the following.

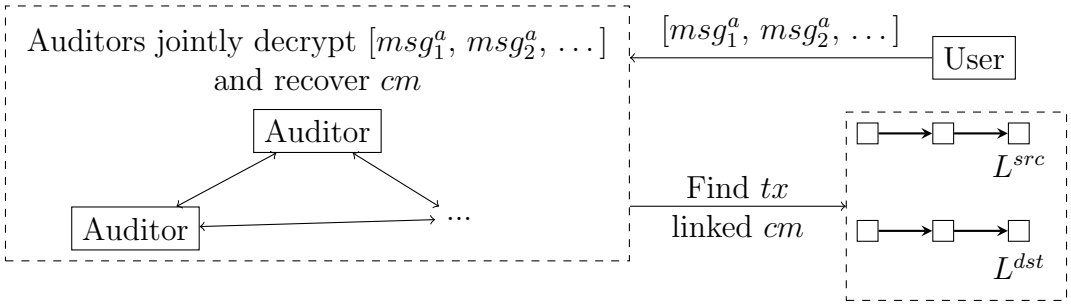

Figure 4: Audit Process

Ledger This protocol is based on a blockchain network. In fact, there are two ledgers: the source chain's ledger L^{src} and the destination chain's ledger L^{dst}. At any given time T, all users have access to $L_T^{\{src,dst\}}$. Both ledgers are append-only.
Public parameters A list of public parameters pp is available to all users in the system. These are generated by a trusted party at the "start of time" and are used by the system's algorithms.
Address Each user generates at least one address key pair $(addr_{pk}, addr_{sk})$ and a nullifier key nk. The public key $addr_{pk}$ is published and enables others to direct payments to the user. The secret key $addr_{sk}$ is used to receive payments sent to $addr_{pk}$. The nullifier key nk is used to generate serial numbers of receiving coins. A user may generate any number of address key pairs.
Auditable keys. Each user generates at least one audit key pair (pk_u^a, sk_u^a). The corresponding public key pk_u^a is then published. Simultaneously, each auditor prepares an audit key pair $(pk_\cdot^a, sk_\cdot^a)$, with the public key $pk_\cdot^a$ also published. Utilizing an elliptic curve integrated encryption scheme, the user derives a shared secret key from sk_u^a and $pk_\cdot^a$, while the auditor derives it from $sk_\cdot^a$ and pk_u^a.
Coin. A coin is a data object c. Across this paper, c refers to a logical coin since a user will not mint a new coin when transferring the coin. A coin is associated with *commitment, value, serial number, and address*.

- commitment, denoted $cm(c)$: a string that appears on the ledger once c is deposited.
- value, denoted $v(c)$: the denomination of c. We limit the value within some pre-defined default values, denoted $\mathbb{V}$, i.e., $v \in \mathbb{V}$.
- serial number, denoted $sn(c)$: a unique string associated with c, used to prevent double withdrawing.
- address, denoted $addr_{pk}(c)$: an address public key, representing who owns c.

Transaction. We introduce three new types of transactions.

- Deposit transactions. A deposit transaction $tx_{deposit}$ is a tuple $(cm, v, *)$, where cm is the coin commitment, v is the coin value, and $*$ denotes other information, e.g., randomness. The transaction $tx_{deposit}$ records that a user deposits a coin with commitment cm and value v, which could be withdrawn on other chains. $tx_{deposit}$ is recorded on L^{src} and synced to L^{dst}.
- Withdraw transactions. A withdraw transaction $tx_{withdraw}$ is a tuple $(\ [(rt, sn)]\ , cm_1^{new}, cm_2^{new}, v^{pub}, ADDR, \pi_{\mathsf{WITHDRAW}}, [msg_1^a, msg_2^a, \dots, msg_n^a], *)$, where $[(rt, sn)]$ is a set of the Merkle root and the serial number for each old coin , cm_1^{new}, cm_2^{new} are commitments of new coins,

v^{pub} is the public coin value, $ADDR$ is a plain text address, $[msg_1^a, msg_2^a, \ldots, msg_n^a]$ are encrypted messages for the audit, and $*$ denotes other information. The transaction $tx_{withdraw}$ records that a user withdraws some coins c and sends a coin to a public address and two new coins to anonymous addresses. It also contains messages that auditors may decrypt and then track the transaction. $tx_{withdraw}$ is recorded on L^{dst}.

- ZK-rollup transactions. A ZK-rollup transaction tx_{rollup} is a tuple $(rt^{old}, rt^{new}, hash_{[cm]}, pathIndices, N^{rollup}, \pi_{\mathsf{ROLLUP}}, *)$, where rt^{old} is the old Merkle tree root, rt^{new} is the new Merkle tree root after updating with coin commitments $[cm]$, $hash_{[cm]}$ is the hash of $[cm]$, $pathIndices$ is the direction selector of the authentication path of $[cm]$, N^{rollup} is the number of commitments been rolled up, and $*$ denotes other information. The direction selector of the authentication path indicates whether to use the left or right sibling hash at each level when traversing a Merkle tree to verify data from a leaf to the root. The transaction tx_{rollup} records that a user updates the commitment tree with a set of deposited coin commitments. tx_{rollup} is recorded on L^{dst}.

Commitments of deposit coins and serial numbers of withdrawn coins. For any given time T

- $CMList_T$ denotes the list of all commitments appearing in deposit transactions in L_T^{src}.
- $SNList_T$ denotes the list of all serial numbers appearing in withdraw transactions in L_T^{src}

Merkle tree over commitments. For any given time T, $Tree_T$ denotes a Merkle tree over $CMList_T$ and rt_T is the root. $Path_T(cm)$ denotes the path function which outputs the authentication path given a coin commitment cm.
Queue of commitments. For any given time T, Q_T^{cm} denotes a queue of commitments waiting for rollup.

4.2. Algorithms

The protocol Π is a tuple of polynomial-time algorithms (Setup, CreateAddress, CreateAuditKey, Deposit, Withdraw, Rollup, VerifyTransaction, Receive, Audit) with the following syntax and semantics.
System setup. The algorithm Setup generates a list of public parameters:

- Inputs: security parameter λ
- Outputs: public parameters pp

The Setup algorithm is executed once by a trusted party.
Creating payment address. The CreateAddress algorithm generates a new pair of payment address and a nullifier key:

- Inputs: public parameters pp
- Outputs:
  - address key pair $(addr_{pk}, addr_{sk})$
  - nullifier key nk

Each user needs to generate at least one address pair. $addr_{pk}$ is public, and $addr_{sk}$ is kept secretly and used to withdraw the coin sent to the address.

Creating audit keys. The CreateAuditKey algorithm generates a new pair of key for the audit:

- Inputs: public parameters pp
- Outputs: address key pair (pk_u^a, sk_u^a)

Each user needs to generate at least one audit key pair. pk_u^a is public, and sk_u^a is kept secretly.

Depositing coins. The Deposit generates a logical coin and a deposit transaction:

- Inputs:
  - public parameters pp
  - coin value $v \in \mathbb{V}$
  - destination address public key $addr_{pk}$
- Outputs:
  - coin c
  - deposit transaction $tx_{deposit}$

The output coin c has value v and coin address $addr_{pk}$; the output deposit transaction $tx_{deposit}$ equals $(cm, v, *)$, where cm is the coin commitment of c.

Withdrawing coins. The Withdraw algorithm transfers value from coins on one chain to coins on another chain.

- Inputs:
  - public parameters pp
  - For each old coin c,
    - the Merkle root rt
    - authentication path $path$ from commitment $cm(c)$ to root rt
    - the address secret key $addr_{sk}$
  - new address $ADDR$
  - public value v^{pub}
  - new values v_1^{new}, v_2^{new}
  - new address public keys $addr_{pk,1}^{new}, addr_{pk,2}^{new}$
  - user's audit key pair (sk_u^a, pk_u^a)
  - auditors' public keys $[pk_{enc}^a]$
- Outputs:
  - withdraw transaction $tx_{withdraw}$
  - new coins c_1^{new}, c_2^{new}

For each coin c, the Withdraw algorithm takes as inputs a coin c and its address secret key $addr_{sk}$. The Withdraw algorithm also takes as inputs the Merkle tree root rt and an authentication path $path$ of the commitment $cm(c)$. $ADDR$ is the new address where the user sends the public coin, which could be on a different chain other than c's. The value v^{pub} specifies the value to be publicly transferred. (sk_u^a, pk_u^a) and $[pk_{enc,i}^a]$ encrypt commitments for the audit. Moreover, the Withdraw algorithm also generates two new anonymous coins c_1^{new}, c_2^{new} with values v_1^{new}, v_2^{new} and recipients address $addr_{pk,1}^{new}, addr_{pk,2}^{new}$ respectively. $v^{pub} + v_1^{new} + v_2^{new}$ should be equal to c's value. $[msg_1^a, msg_2^a, \ldots, msg_n^a]$ are encrypted commitments.

The Withdraw algorithm outputs a withdraw transaction $tx_{withdraw}$. The transaction $tx_{withdraw}$ equals ($[(rt, sn)]$, $cm_1^{new}, cm_2^{new}, v^{pub}, ADDR$, π_{WITHDRAW}, $[msg_1^a, msg_2^a, \ldots, msg_n^a]$). This transaction will not reveal the payment address of the old coin.

ZK-rollup. The algorithm Rollup generates a new Merkle tree root and a ZK-rollup transaction:

- Inputs:
  - public parameters pp
  - rollup size N^{rollup}
  - a queue of deposited commitments Q^{cm}
  - an old Merkle tree root rt^{old}
  - an authentication path $path$
- Outputs:
  - a set of deposited commitments $[cm]$
  - ZK-rollup transaction tx_{rollup}

The Rollup algorithm takes as inputs an old Merkle root rt^{old}, an authentication path $path$, a rollup size N^{rollup}, and a queue of deposited commitments Q^{cm}. The Rollup algorithm outputs a set of deposit commitments $[cm]$ by dequeuing N^{rollup} commitments from Q^{cm}. It also generates a new Merkle root rt^{new} by updating leaves in the old Merkle tree with new leaves $[cm]$. There is an authentication path $path$ toward the ancestor node of new leaves, which is equal to the root of a CRH-based Merkle tree over $[cm]$. The algorithm then generates a zk-SNARK π_{ROLLUP} to prove that all calculations are valid and correct. The output ZK-rollup transaction tx_{rollup} equals $(rt^{old}, rt^{new}, hash_{[cm]}, pathIndices, N^{rollup}, \pi_{\mathsf{ROLLUP}}, *)$, where $hash_{[cm]}$ is the hash of $[cm]$, $pathIndices$ is the direction selector of $path$.

Verifying transactions. The algorithm VerifyTransaction checks the validity of a transaction:

- Inputs:
  - public parameters pp
  - a (withdraw, deposit or ZK-rollup) transaction tx
  - the current source and destination ledgers L^{src} and L^{dst}
- Outputs: bit $\mathcal{B}$, equals 1 iff the transaction is valid

Deposit, withdraw, and ZK-rollup transactions must be verified before being executed.
Receiving coins.[1] The algorithm Receive scans the ledger and retrieves unwithdrawn coins paid to a particular user address:

- Inputs:
  - recipient address key pair $(addr_{pk}, addr_{sk})$
  - recipient nullifier address nk
  - the current source and destination ledgers L^{src} and L^{dst}
- Outputs: set of (unwithdrawn) received coins

When a user with address key pair $(addr_{pk}, addr_{sk})$ wishes to receive payments sent to $addr_{pk}$, the user uses the Receive algorithm to scan the ledger. For each payment to $addr_{pk}$ appearing in the ledger, Receive outputs the corresponding coins whose serial numbers do not appear on the ledger L^{src} or L^{dst}. Coins received in this way may be withdrawn by using the Withdraw algorithm.
Audit. The algorithm Audit audits user transactions:

- Inputs:
  - Encrypted commitments sharings $[msg_1^a, msg_2^a, \ldots, msg_n^a]$
  - User's public key pk_u^a
  - Auditors' private keys $[sk_{enc,1}^a, sk_{enc,2}^a, \ldots, sk_{enc,n}^a]$
- Outputs: A set of commitments $[cm]$

The auditors decrypt each message msg_i^a with a shared secret key $sk_{enc,i}^a$, pk_u^a and jointly recover the commitments $[cm]$. The auditors can recover the transactions linked with those commitments.

4.3. Completeness

Completeness of a protocol requires that unwithdrawn coins can be withdrawn. Suppose a ledger sampler $\mathcal{S}$ outputs a ledger L^{src} or L^{dst} . If c is a coin whose commitment appears in a valid transaction on L^{src} or L^{dst} , but its serial number does not appear in L, then c can be withdrawn using Withdraw transaction. Informally, if Withdraw outputs a $tx_{withdraw}$ transaction that VerifyTransaction accepts, the coin could be received by the intended recipient. Raikwar et al. [42] introduced a similar concept, termed **Overdraft Safety**. This property is formalized via an *incompleteness experiment* $INCOMP$ 6.1.

Definition 1. *A protocol* Π=(Setup, CreateAddress, CreateAuditKey, Deposit, Withdraw, Rollup, VerifyTransaction, Receive, Audit*) is complete if no polynomial-size ledger sample* $\mathcal{S}$ *wins* $INCOMP$ *with more than negligible probability.*

[1] Taken from [1] 3.2 **Receiving coins**

4.4. Security

Security of the protocol is characterized by four properties, which we call ledger *indistinguishability*, *transaction non-malleability*, *balance*, and *auditability*.

Definition 2. *A protocol* Π=(Setup, CreateAddress, CreateAuditKey, Deposit, Withdraw, Rollup, VerifyTransaction, Receive, Audit) *is secure if it satisfies ledger indistinguishability, transaction non-malleability, balance, and auditability.*

We provide some informal definitions in the following.
Ledger indistinguishability. This property captures the requirement that the ledger reveals no new information to the adversary beyond the publicly-revealed information (e.g. plain text address, coin's public value).
Transaction non-malleability. This property means that no bounded adversary can modify the data stored in a valid withdraw transaction.
Balance. This property requires that no bounded adversary could withdraw more coins than what the user received from the deposit transaction.
Auditability. The protocol does not disclose users' transaction data unless a predefined threshold of auditors reaches consensus. This threshold is configurable and can be tailored based on deployment requirements. For example, a system may require a two-thirds majority approval or a simple majority of auditors to access encrypted transaction details. By making the threshold flexible, the protocol ensures a balance between privacy protection and regulatory compliance.

5. Construction of the Protocol

In this section, we initially outline the construction of the protocol using zk-SNARK and other cryptographic building blocks, followed by a detailed description of the concrete design.

5.1. Cryptographic building blocks

We introduce the formal notation of the cryptography building blocks we use. λ denotes the security parameter. This part is similar to [1] **Section 4.1**.
Collision-resistant hashing. We use a collision-resistant hash function $CRH : \{0,1\}^* \to \{0,1\}^{O(\lambda)}$.
Pseudorandom functions. We use a pseudorandom function family $\mathbb{PRF} = \{PRF_x : \{0,1\}^* \leftarrow \{0,1\}^{O(\lambda)}\}_x$. x is the seed that initializes the PRF. We then instance three pseudorandom functions from the same $PRF_x s \xleftarrow{\$} \mathbb{PRF}$ and add different prefix to the input. Namely, $PRF_x^{addr}(z) := PRF_x(00||z)$, $PRF_x^{sn}(z :) = PRF_x(01||z)$, $PRF_x^{pk}(z) := PRF_x(10||z)$. Moreover, we require PRF^{sn} to be collision resistant, i.e. one cannot find $(x,z) \neq (x',z')$ s.t. $PRF_x^{sn}(z) = PRF_{x'}^{sn}(z')$.
Statistically-hiding commitments. We use a computationally binding and statistically hiding commitment scheme $COMM$. Namely, $\{COMM_x : \{0,1\}^* \to \{0,1\}^{O(\lambda)}\}_x$ where x denotes the trapdoor parameter.
One-time strongly-unforgeable digital signatures. We use a digital signature scheme $Sig = (G_{sig}, K_{sig}, S_{sig}, V_{sig})$.

- $G_{sig}(1^\lambda) \to pp_{sig}$. Given a security parameter λ, G_{sig} samples public parameters pp_{sig} for the signature scheme.
- $K_{sig}(pp_{sig}) \to (pk_{sig}, sk_{sig})$. Given public parameters pp_{sig}, K_{sig} samples a public key and a secret key for a single user.

- $S_{sig}(sk_{sig}, m) \rightarrow \sigma$. Given a secret key sk_{sig} and a message m, S_{sig} signs m to obtain a signature σ.
- $V_{sig}(pk_{sig}, m, \sigma) \rightarrow b$. Given a public key pk_{sig}, message m, and the signature σ, V_{sig} outputs $b = 1$ if validated or otherwise $b = 0$.

We require Sig to be one-time strong unforgeable against chosen-message attacks (SUF-1CMA security).

Key-private public-key encryption. We use a public-key encryption scheme $Enc = (G_{enc}, K_{enc}, E_{enc}, D_{enc})$.

- $G_{enc}(1^\lambda) \rightarrow pp_{enc}$. Given a security parameter λ, G_{enc} samples public parameters pp_{enc} for the encryption scheme.
- $K_{enc}(pp_{enc}) \rightarrow (pk_{enc}, sk_{enc})$. Given public parameters pp_{enc}, K_{enc} samples a public key and a secret key for a single user.
- $E_{enc}(pk_{enc}, m) \rightarrow Ct$. Given a public key pk_{enc} and a message m, E_{enc} encrypts m to obtain a cipher text Ct.
- $D_{enc}(sk_{enc}, Ct) \rightarrow m$. Given a secret key sk_{enc} and a cipher text Ct, D_{enc} decrypts Ct to obtain the plain message m (or $\perp$ if decryption fails).

The encryption scheme Enc is secure against chosen-ciphertext attack and provides ciphertext indistinguishability IND-CCA and key indistinguishability IK-CCA.

Elliptic curve integrated encryption scheme. We use an elliptic curve integrated encryption scheme $ECIES = (G_{ecies}, K_{ecies}, KEM, SEC.Enc, SEC.Dec)$.

- $G_{ecies}(1^\lambda) \rightarrow pp_{ecies}$. Given a security parameter λ, G_{ecies} samples public parameters pp_{ecies} for the encryption scheme.
- $K_{ecies}(pp_{ecies}) \rightarrow (pk^a, sk^a)$. Given public parameters pp_{ecies}, K_{enc} samples a public key and a secret key for a single user.
- $KEM(pk_i^a, sk_j^a) \rightarrow k^a$. Given a public key from user i and a private key from user j, KEM generates a shared secret key k^a.
- $SEC.Enc_{k^a}(m) \rightarrow msg^a$. Given a secret key k^a and a message m, $SEC.Enc$ encrypts m to obtain a ciphered text msg^a.
- $SEC.Dec_{k^a}(msg^a) \rightarrow m$. Given a secret key sk_{enc} and a cipher text msg^a, $SEC.Dec$ decrypts msg^a to obtain the plain message m (or $\perp$ if decryption fails).

The encryption scheme $ECIES$ is secure against chosen-ciphertext attack and provides ciphertext indistinguishability IND-CCA and key indistinguishability IK-CCA.

Threshold secret sharing. We use a threshold secret sharing scheme $SS = (Share, Recover)$.

- $Share(x) \rightarrow [x_1, x_2, \ldots, x_n]$. Given a secret x generates n secret shares $[x_1, x_2, \ldots, x_n]$.
- $Recover([x_i, x_{i+1}, \ldots, x_{i+t-1}]) \rightarrow x$. Given t secret shares $[x_i, x_{i+1}, \ldots, x_{i+t-1}]$ generates the secret x.

The secret sharing is a perfect t out of n secret sharing PER-SS, i.e., the secret sharing scheme outputs n shares, and given any t shares, we can recover the secret. We learn nothing about x given less than t shares.

5.2. zk-SNARKs for withdrawing coins

We use zk-SNARK to prove an NP statement $WITHDRAW$. For the definition of zk-SNARK, we refer to [10] for a detailed explanation. We first give an informal definition of zk-SNARKs. Given a field $\mathbb{F}$, a **zk-SNARK** for $\mathbb{F}-$arithmetic circuit satisfiability is a triple of polynomial-time algorithm $(\mathsf{KeyGen}, Prove, Verify)$:

- $\mathsf{KeyGen}(1^\lambda, C) \to (pk, vk)$. On input of a security parameter λ and an $\mathbb{F}-$arithmetic circuit C, the *key generator* KeyGen probabilistically samples a *proving key* **pk** and a *verification key* **vk**.

- $Prove(pk, x, a) \to \pi$. On input a proving key **pk** and any $(x, a) \in R_C$, the *prover* **Prove** outputs a non-interactive proof π for the statement $x \in L_C$. **Relation** R_C: The relation denotes the relation corresponding to the circuit C. The relation R_C is a set of valid pairs (x, a), where x is the statement, and a is the witness.

  $$R_C = \{(x, a) \mid C(x, a) = 1\}$$

  Language L_C: The language contains all valid statements x for which there exists a witness a that satisfies the relation R_C.

  $$L_C = \{x \mid \exists a \text{ such that } (x, a) \in R_C\}$$

- $Verify(vk, x, \pi) \to b$. On input a verification key **vk**, an input x, and a proof π, the *verifier* **Verify** outputs $b = 1$ if the *verifier* is convinced that $x \in L_C$.

We recall the corresponding withdraw transaction $tx_{withdraw} = (\ [(rt, sn)]\ , cm_1^{new}, cm_2^{new}, v^{pub}, ADDR, \pi_{\mathsf{WITHDRAW}}, [msg_1^a, msg_2^a, \ldots, msg_n^a])$. To withdraw a coin c, a user u should show that

1. u owns c
2. commitment of c appears on the ledger
3. sn is the calculated correctly as the serial number of c
4. balance is preserved
5. the commitment is well encrypted

which is formalized as a statement $WITHDRAW$ and proved with zk-SNARK. We then define the statement as follows.

- Instances is $x := ([(rt, sn, h)], v^{pub}, cm_1^{new}, cm_2^{new}, h_{sig},\ pk_u^a,\ [pk_{enc}^a],\ [msg^a])$, which specifies a set $[(rt,\ sn, h)]$ for each old coin, where rt is the root for a CRH-based Merkle tree, sn is the serial number, and h is the signature. It also specifies the public value v^{pub}, two commitments of new coins cm_1^{new}, cm_2^{new}, and fields h_{sig} used for non-malleability. pk_u^a is the user's private key for audit, pk_{enc}^a are auditors' public keys, and $[msg^a]$ are encrypted secret sharings of commitments for audit.

- Witnesses are of the form $a := ([(path, c, addr_{sk})], c_1^{new}, c_2^{new}, [cm^a], sk_u^a)$ where

$$\begin{aligned} c &= (addr_{pk}, v, \rho, r, s, cm) \\ addr_{pk} &= (a_{pk}, pk_{enc}) \\ c_i^{new} &= (addr_{pk,i}^{new}, v_i^{new}, \rho_i^{new}, r_i^{new}, s_i^{new}, cm_i^{new}) \\ addr_{pk,i}^{new} &= (a_{pk,i}^{new}, pk_{enc,i}^{new}) \end{aligned}$$

  The notations in c are defined in Algorithm **??**. Thus, the witness a specifies an authenticated path from root rt to the coin's commitment, the entirety information of the coin c, the address secret key, secret sharings of commitments, and the user's private key for audit.

Given a $WITHDRAW$ instance x, a witness a is valid for x if :

1. For any old coin c,
   (a) The coin's commitment cm appears on the ledger, i.e., $path$ is a valid authentication path for leaf cm in a CRH-based Merkle tree with root rt.
   (b) The address secret key a_{sk} matches the address public key, i.e., $a_{pk} = PRF_{a_{sk}}^{addr}(0)$.
   (c) The nullifier key nk matches the address secret key, i.e., $nk = PRF_{a_{sk,1}}^{addr}(1)$.
   (d) The serial number sn is computed correctly, i.e., $sn = PRF_{nk}^{sn}(\rho)$.
   (e) The coin c is well formatted, i.e., $cm = COMM_s(COMM_r(a_{pk}||\rho)||v)$.
   (f) The address secret key a_{sk} ties to h_{sig} to h, i.e., $h = PRF_{a_{sk}}^{pk}(h_{sig})$.
2. New coins c_1^{new} and c_2^{new} are well formatted, i.e., $cm = COMM_{s_i^{new}}(COMM_{r_i^{new}}(a_{pk,i}^{new}||\rho_i^{new})||v_i^{new})$.
3. Balance is preserved, i.e. $\sum v = v_1^{new} + v_2^{new} + v^{pub}$.
4. The commitments are well secret shared, i.e. $[cm^a] = Share^{(t,n)}([cm])$.
5. The public audit key matches the private audit key, i.e., $pk_u^a = sk_u^a G$.
6. Each commitments share is well encrypted, i.e., $msg_i^a = SEC.Enc_{sk_u^a pk_{enc,i}^a}(cm_i^a)$.

The reason for sharing commitments instead of the actual coins or values is that commitments are the only data recorded on the ledger. Ensuring that the public audit key matches the corresponding private audit key is crucial because it guarantees that the user has employed the correct key pair. This match is necessary so that auditors can later decrypt the encrypted values reliably. In essence, it confirms the authenticity of the encryption process and ensures that the encrypted data can be properly audited in the future.

5.3. zk-SNARKs for ZK-rollup

We use zk-SNARK to prove an NP statement $ROLLUP$. In this section, we use the same notions as in Section 5.2. We recall the corresponding ZK-rollup transaction $tx_{rollup} = (rt^{old}, rt^{new}, hash_{[cm]}, pathIndices, N^{rollup}, \pi_{\mathsf{ROLLUP}})$. To rollup a set of coin commitments $[cm]$, a user u should show that

1. u knows $[cm]$
2. u updates the old Merkle tree with $[cm]$

which is formalized as a statement $ROLLUP$ and proved with zk-SNARK. We then define the statements as follows.

- Instances is $x := (rt^{old}, rt^{new}, hash_{[cm]}, pathIndices, N^{rollup})$, which specifies an old Merkle root rt^{old}, a new Merkle root rt^{new}, a hash of a set of coin commitments $hash_{[cm]}$, the direction selector of the updated leaf's authentication path $pathIndices$.
- Witnesses are of the form $a := ([cm], path)$, and the rollup size N^{rollup}.

Thus, the witness a specifies a set of commitments $[cm]$, and the authentication path $path$.

Given a $ROLLUP$ instance x, let $[0]$ be a set of N^{rollup} zeros, a witness a is valid for x if :

1. $hash_{[cm]}$ is the hash value of $[cm]$.
2. $rt^{[0]}$ is the Merkle root of $[0]$.
3. $path$ is a valid authentication path from $rt^{[0]}$ to rt^{old}, and the corresponding director selector is $pathIndices$.
4. $rt^{[cm]}$ is the root of a CRH-based Merkle tree over $[cm]$.
5. $path$ is a valid authentication path from $rt^{[cm]}$ to rt^{new}, and the corresponding director selector is $pathIndices$.
6. The number of updated leaves is correct, e.g., let H be the height of the whole Merkle tree, $|[cm]| = |[0]|$ and $|path| + \log_2 |[cm]| - 1 = H$.

5.4. Algorithm constructions

In this section, we describe the construction of each algorithm. The intuition is given in Sections 4.1 and 4.2. The building blocks are introduced in 5.1 and 5.2. We give the pseudocode for each algorithm in Figure 5 and Figure 6.

5.5. Incentive Model for ZK-Rollup Miners

Mystiko ZK-rollup encourages miners to collect transactions, generate proofs, and submit them to the blockchain. To ensure active participation and honest behavior, the protocol provides the following incentives:

- **Rewards for Proof Submission:** Miners receive a reward for successfully submitting a valid rollup proof. This reward comes from small fees paid by users who submit transactions.
- **Efficiency Incentive:** Since a single proof can bundle multiple transactions, miners benefit from processing more transactions together, reducing costs for everyone.
- **Penalty for Incorrect Proofs:** If a miner submits an invalid proof, the system rejects it, and the miner does not receive any reward. Repeated failures can lead to the miner being temporarily banned from submitting new proofs.

This incentive structure ensures that miners are motivated to participate correctly and efficiently, keeping transaction costs low while maintaining the security and reliability of the system.

Setup.

- Inputs: security parameter λ
- Outputs: public parameters pp

1. Construct the arithmetic circuit C_{WITHDRAW} for the $WITHDRAW$ statement at security λ.
2. Compute $(pk_{\mathsf{WITHDRAW}}, vk_{\mathsf{WITHDRAW}}) := \mathsf{KeyGen}(1^\lambda, C_{\mathsf{WITHDRAW}})$.
3. Construct the arithmetic circuit C_{ROLLUP} for the $ROLLUP$ statement at security λ.
4. Compute $(pk_{\mathsf{ROLLUP}}, vk_{\mathsf{ROLLUP}}) := \mathsf{KeyGen}(1^\lambda, C_{\mathsf{ROLLUP}})$.
5. Compute $pp_{enc} := G_{enc}(1^\lambda)$.
6. Compute $pp_{sig} := G_{sig}(1^\lambda)$.
7. Compute $pp_{ecies} := G_{ecies}(1^\lambda)$.
8. Set $pp := (pk_{\mathsf{WITHDRAW}}, vk_{\mathsf{WITHDRAW}}, pk_{\mathsf{ROLLUP}}, vk_{\mathsf{ROLLUP}}, pp_{enc}, pp_{sig}, pp_{ecies})$.
9. Output pp.

CreateAddress

- Inputs: public parameters pp
- Outputs:
  - address key pair $(addr_{pk}, addr_{sk})$
  - nullifier key nk

1. Compute $(pk_{enc}, sk_{enc}) := K_{enc}(pp_{enc})$.
2. Randomly sample a PRF^{addr} seed a_{sk}.
3. Compute $a_{pk} = PRF^{addr}_{a_{sk}}(0)$.
4. Compute $nk = PRF^{addr}_{a_{sk}}(1)$.
5. Set $addr_{pk} := (a_{pk}, pk_{enc})$.
6. Set $addr_{sk} := (a_{sk}, sk_{enc})$.
7. Output $(addr_{pk}, addr_{sk})$ and nk.

CreateAuditKey

- Inputs: public parameters pp
- Outputs: address key pair (pk^a_u, sk^a_u)

1. Compute $(pk^a_u, sk^a_u) := K_{ecies}(pp_{ecies})$.
2. Outputs (pk^a_u, sk^a_u).

Deposit

- Inputs:
  - public parameters pp
  - coin value $v \in \mathbb{V}$
  - destination address public key $addr_{pk}$
- Outputs:
  - coin c
  - deposit transaction $tx_{deposit}$

1. Parse $addr_{pk}$ as (a_{pk}, pk_{enc}).
2. Randomly sample a PRF^{sn} seed ρ.
3. Randomly sample two $COMM$ trapdoors r, s.
4. Compute $k := COMM_r(a_{pk}||\rho)$.
5. Compute $cm := COMM_s(v||k)$.
6. Compute $Ct := E_{enc}(pk_{enc}, m)$, where $m := (v, \rho, r, s)$.
7. Set $c := (addr_{pk}, v, \rho, r, s, cm)$.
8. Set $tx_{deposit} := (cm, v, *)$, where $* := (k, s, Ct)$.
9. Output c and $tx_{deposit}$.

Audit.

- Inputs:
  - Encrypted commitments sharings $[msg^a_1, \ldots, msg^a_n]$
  - User's public key pk^a_u
  - Auditors' private keys $[sk^a_{enc,1}, sk^a_{enc,2}, \ldots, sk^a_{enc,n}]$
- Outputs: A set of commitments $[cm]$

1. For each auditors' private key $sk^a_{enc,i}$, compute $k^a_i := KEM(pk^a_u, sk^a_{enc,i})$.
2. For each encrypted commitments sharing msg^a_i, compute $cm^a_i := SEC.Dec_{k^a_i}(msg^a_i)$.
3. Compute $[cm] := Recover^{(t,n)}(cm^a_1, cm^a_2, \ldots, cm^a_n)$
4. Output $[cm]$.

Withdraw.

- Inputs:
  - public parameters pp
  - For each coin c,
    - the Merkle root rt
    - authentication path $path$ from commitment $cm(c)$ to root rt
    - the address secret key $addr_{sk}$
    - nullifier key nk
  - new address $ADDR$
  - public value v^{pub}
  - new values v^{new}_1, v^{new}_2
  - new address public keys $addr^{new}_{pk,1}, addr^{new}_{pk,2}$
  - user's audit key pair (sk^a_u, pk^a_u)
  - auditors' public keys $pk^a_{enc,1}, pk^a_{enc,2}, pk^a_{enc,3}$
- Outputs:
  - withdraw transaction $tx_{withdraw}$
  - new coins c^{new}_1, c^{new}_2

1. For each old coin c:
   (a) Parse c as $(addr_{pk}, v, \rho, r, s, cm)$.
   (b) Parse $addr_{sk}$ as (a_{sk}, sk_{enc}).
   (c) Compute $sn := PRF^{sn}_{nk}(\rho)$.
   (d) Parse $addr_{pk}$ as (a_{pk}, pk_{enc}).
2. For each $i \in 1, 2$:
   (a) Parse $addr^{new}_{pk,i}$ as $(a^{new}_{pk,i}, pk^{new}_{enc,i})$.
   (b) Randomly sample a PRF^{sn} seed ρ^{new}_i.
   (c) Randomly sample two $COMM$ trapdoors r^{new}_i, s^{new}_i.
   (d) Compute $k^{new}_i := COMM_{r^{new}_i}(a^{new}_{pk,i}||\rho^{new}_i)$.
   (e) Compute $cm^{new}_i := COMM_{s^{new}_i}(v^{new}_i||k^{new}_i)$.
   (f) Compute $Ct^{new}_i := E_{enc}(pk_{enc}, m)$, where $m := (v^{new}_i, \rho^{new}_i, r^{new}_i, s^{new}_i)$.
   (g) Set $c^{new}_i := (addr^{new}_{pk,i}, v^{new}_i, \rho^{new}_i, r^{new}_i, s^{new}_i, cm^{new}_i)$.
3. Generate $(pk_{sig}, sk_{sig}) := K_{sig}(pp_{sig})$.
4. Compute $h_{sig} := CRH(pk_{sig})$.
5. For each old coin, compute $h := PRF^{pk}_{a_{sk}}(i||h_{sig})$.
6. Compute $[cm^a_1, cm^a_2, \ldots, cm^a_n] := Share^{(t,n)}([cm])$.
7. For each auditor's public key $pk^a_{enc,i}$, compute $k^a_i = KEM(pk^a_{enc,i}, sk^a_u)$.
8. For each commitments share cm^a_i, compute $msg^a_i := SEC.Enc_{k^a_i}(cm^a_i)$.
9. Set $x := ([(rt, sn, h)], v^{pub}, cm^{new}_1, cm^{new}_2, h_{sig}, pk^a_u, pk^a_{enc,1}, pk^a_{enc,2}, pk^a_{enc,3}, [msg^a_1, msg^a_2, \ldots, msg^a_n])$.
10. Set $a = ([(path, c, addr_{sk})], c^{new}_1, c^{new}_2, cm^a_1, cm^a_2, cm^a_3, sk^a_u)$.
11. Compute $\pi_{\mathsf{WITHDRAW}} := Prove(pk_{\mathsf{WITHDRAW}}, x, a)$.
12. Set $m := (x, \pi_{\mathsf{WITHDRAW}}, ADDR, Ct^{new}_1, Ct^{new}_2)$.
13. Compute $\sigma := S_{sig}(sk_{sig}, m)$.
14. Set $tx_{withdraw} = ([(rt, sn)], cm^{new}_1, cm^{new}_2, v^{pub}, ADDR, \pi_{\mathsf{WITHDRAW}}, msg^a_1, msg^a_2, msg^a_3, *)$, where $* := (pk_{sig}, [h], \sigma, Ct^{new}_1, Ct^{new}_2)$.
15. Output c^{new}_1, c^{new}_2, and $tx_{withdraw}$.

Figure 5: Construction of Setup, CreateAddress, CreateAuditKey, Deposit, Audit, Withdraw algorithms.

Rollup.

- Inputs:
  - public parameters pp
  - rollup size N^{rollup}
  - a queue of deposited commitments Q^{cm}
  - an old Merkle tree root rt^{old}
  - an authentication path $path$
- Outputs:
  - a set of deposited commitments $[cm]$
  - ZK-rollup transaction tx_{rollup}

1. Set $pathIndices$ as the direction selector of $path$.
2. Set $[cm]$ as the first N^{rollup} commitments from Q^{cm}.
3. Compute $hash_{[cm]} := CRH([cm])$.
4. Compute $rt^{[cm]}$ as the root of a CRH-based Merkle tree over $[cm]$.
5. Compute rt^{new} as follows:
   (a) Let D^{path} be the length of $path$.
   (b) Let $digest = rt^{[cm]}$.
   (c) For each $i \in \{1, \ldots, D^{path}\}$, if $pathIndices[i] = 0$, compute $digest := CRH(digest, path[i])$, else $digest := CRH(path[i], digest)$.
   (d) Set $rt^{new} := digest$
6. Set $x := (rt^{old}, rt^{new}, hash_{[cm]}, pathIndices, N^{rollup})$.
7. Set $a := ([cm], path)$.
8. Compute $\pi_{\mathsf{ROLLUP}} := Prove(pk_{\mathsf{ROLLUP}}, x, a)$.
9. Set $tx_{rollup} := (rt^{old}, rt^{new}, hash_{[cm]}, pathIndices, N^{rollup}, \pi_{\mathsf{ROLLUP}})$.
10. Ouput $[cm]$ and tx_{rollup}.

Receive.

- Inputs:
  - recipient address key pair $(addr_{pk}, addr_{sk})$
  - recipient nullifier key nk
  - the current source and destination ledgers L_{src} and L_{dst}
- Outputs: set of (unwithdrawn) received coins

1. Parse $addr_{pk}$ as (a_{pk}, pk_{enc}).
2. Parse $addr_{sk}$ as (a_{sk}, sk_{enc}).
3. For each deposit transaction $tx_{deposit}$ on the ledger:
   (a) Parse $tx_{deposit}$ as $(cm, v, *)$, where $*$ as (k, s, Ct).
   (b) Compute $m := D_{enc}(sk_{enc}, Ct)$, and parse m as (v, ρ, r, s).
   (c) If D_{enc}'s output is not $\perp$, verify that:
   - cm equals $COMM_s(v||COMM_r(a_{pk}||\rho))$;
   - $sn := PRF^{sn}_{nk}$ does not appear on L.

   (d) If both checks succeed, output $c := (addr_{pk}, v, \rho, r, s, cm)$

VerifyTransaction.

- Inputs:
  - public parameters pp
  - a (withdraw or deposit) transaction tx
  - auditors' public keys $[pk^a_{enc,1}, pk^a_{enc,2}, \ldots, pk^a_{enc,n}]$
  - the current source and destination ledgers L_{src} and L_{dst}
- Outputs: bit b, equals 1 iff the transaction is valid

1. If given a deposit transaction $tx = tx_{deposit}$:
   (a) Parse $tx_{deposit}$ as $(cm, v, *)$, and $*$ as (k, s).
   (b) If $v \notin \mathbb{V}$, output $b := 0$.
   (c) Set $cm' := COMM_s(v||k)$.
   (d) Output $b := 1$ if $cm = cm'$, else output $b := 0$.
2. If given a withdraw transaction $tx = tx_{withdraw}$:
   (a) Parse $tx_{withdraw}$ as $([(rt, sn)], cm_1^{new}, cm_2^{new}, v^{pub}, ADDR, \pi_{\mathsf{WITHDRAW}}, [msg_1^a, msg_2^a, \ldots, msg_n^a], *)$, where $* := (pk_{sig}, [h], \pi_{\mathsf{WITHDRAW}}, \sigma, Ct_1^{new}, Ct_2^{new})$..
   (b) If any sn appears on L, output $b := 0$.
   (c) If any Merkle root rt does not appear on L, output $b := 0$.
   (d) Compute $h_{sig} := CRH(pk_{sig})$.
   (e) Set $x := ([(rt, sn, h)], v^{pub}, cm_1^{new}, cm_2^{new}, h_{sig}, pk^a_u, [pk^a_{enc,1}, pk^a_{enc,2}, \ldots, pk^a_{enc,n}], [msg_1^a, msg_2^a, \ldots, msg_n^a])$.
   (f) Set $m := (x, \pi_{\mathsf{WITHDRAW}}, ADDR, Ct_1^{new}, Ct_2^{new})$.
   (g) Compute $b := V_{sig}(pk_{sig}, m, \sigma)$.
   (h) Compute $b' := Verify(vk_{\mathsf{WITHDRAW}}, x, \pi_{\mathsf{WITHDRAW}})$, and output $b \wedge b'$.
3. If given a ZK-rollup transaction $tx = tx_{rollup}$:
   (a) Parse tx_{rollup} as $(rt^{old}, rt^{new}, hash_{[cm]}, pathIndices, N^{rollup}, \pi_{\mathsf{ROLLUP}})$
   (b) If rt^{old} does not appear on L, output $b := 0$.
   (c) If rt^{new} appears on L, output $b := 0$.
   (d) If $N^{rollup} <= 0$ or $N^{rollup} > |Q^{cm}|$, output $b := 0$.
   (e) Set $x := (rt^{old}, rt^{new}, hash_{[cm]}, pathIndices, N^{rollup})$.
   (f) Compute $b := Verify(vk_{ROOLUP}, x, \pi_{\mathsf{ROLLUP}})$, and output b

Figure 6: Construction of Rollup, Receive, VerifyTransaction algorithms.

5.6. Concrete design

In this section, we describe how we instantiate each building block. Namely, we build CRH, PRF, $COMM$ from **Poseidon** [43], Merkle tree from **Keccak**[44], Sig from **ECDSA**, Enc from **key-private Elliptic-Curve Integrated Encryption Scheme**.

6. Completeness and Security of the Protocol

In this section, we give a formal definition of the completeness and security of the protocol and our main theorem. We then prove the theorem.

Theorem 1. *The tuple* Π=(Setup, CreateAddress, CreateAuditKey, Deposit, Withdraw, Rollup, VerifyTransaction, Receive, Audit) *is complete and secure.*

6.1. Completeness

In this part, we formally define the completeness of the protocol.

Definition 3. *A protocol* Π=(Setup, CreateAddress, CreateAuditKey, Deposit, Withdraw, Rollup, VerifyTransaction, Receive, Audit) *is complete if for every polynomial-size ledger sample* $\mathcal{S}$ *and sufficiently large* λ, $Adv^{INCOMP}_{\Pi,S}(\lambda) < negl(\lambda)$, *where* $Adv^{INCOMP}_{\Pi,S}(\lambda) := Pr[INCOMP(\Pi, S, \lambda) = 1]$ *is* $\mathcal{S}$*'s advantage in the incompleteness experiment.*

We now describe the incompleteness experiment $INCOMP$. This experiment is an interactive game between a ledger sampler $\mathcal{S}$ and a challenger $\mathcal{C}$. At the beginning, $\mathcal{C}$ samples public parameters $pp \leftarrow \mathsf{Setup}(1^\lambda)$ and sends to $\mathcal{S}$. $\mathcal{S}$ then samples a ledger L and sends back to $\mathcal{C}$. $\mathcal{C}$ also sends a set of coins $[c]$ and parameters for a withdraw transaction, i.e., secret address key $addr_{sk}$, public value v^{pub}, new coin values v_1^{new}, v_2^{new}, new address public keys $addr_{pk,1}^{new}, addr_{pk,2}^{new}$, user's audit key pair (sk_u^a, pk_u^a), auditors' public keys $[pk_{enc}^a]$, and plain text address $ADDR$. Assume, without loss of generality, $\mathcal{C}$ sends two coins c_1, c_2, and there are three auditors. After receiving a message, $\mathcal{C}$ checks validations on $\mathcal{S}$'s message.

Firstly, $\mathcal{C}$ checks if c_1, c_2 are valid coins, i.e. they are well formatted as defined in Section 4. Then, $\mathcal{C}$ checks that values are balanced, i.e. $v_1 + v_2 = v_1^{new} + v_2^{new} + v^{pub}$. $\mathcal{C}$ aborts and outputs 0 if any checks fail.

Otherwise, $\mathcal{C}$ calculates a withdraw transaction with the following steps:

1. Compute the Merkle tree root rt over all coin commitments in L
2. Compute authenticated paths from c_1's commitment cm_1 and c_2's commitment cm_2 to rt
3. Compute $tx_{withdraw} \leftarrow \mathsf{Withdraw}(pp, rt, path_1, path_2, addr_{sk,1}, addr_{sk,2}, v^{pub}, v_1^{new}, v_2^{new}, addr_{pk,1}^{new}, addr_{pk,2}^{new}, (sk_u^a, pk_u^a), pk_{enc,1}^a, pk_{enc,2}^a, pk_{enc,3}^a, ADDR)$

Finally, $\mathcal{C}$ outputs 1 iff following cases hold:

- $tx_{withdraw} \neq (rt, sn_1, sn_2, cm_1^{new}, cm_2^{new}, v^{pub}, ADDR, \pi_{\mathsf{WITHDRAW}}, msg_1^a, msg_2^a, msg_3^a, *)$, or
- $tx_{withdraw}$ is not valid, i.e. $\mathsf{VerifyTransaction}(pp, tx_{withdraw}, L_{src,dst})$ outputs 0.

6.2. Security

In this section, we formally define the three secure properties: ledger indistinguishability, transaction non-malleability, and balance. All properties are defined as interaction games between an adversary $\mathcal{A}$ and a challenger $\mathcal{C}$. We also introduce an oracle $\mathcal{O}^{PRO}$ to simulate the behavior of honest parties. We first describe $\mathcal{O}^{PRO}$ as follows.

$\mathcal{O}^{PRO}$ initially stores a ledger L^{PRO}, a set of address $ADDR^{PRO}$, a set of user's audit key AUD^{PRO}, a set of coins $COIN^{PRO}$, and they all start out empty. $\mathcal{O}^{PRO}$ supports different queries, denoted as $\mathcal{Q}$, as described below:

- $\mathcal{Q} = (\mathsf{CreateAddress})$
  - Compute $(addr_{pk}, addr_{sk}) := \mathsf{CreateAddress}(pp)$.
  - Add the address pair $(addr_{pk}, addr_{sk})$ to $ADDR^{PRO}$.
  - Output the address public key $addr_{pk}$
- $\mathcal{Q} = (\mathsf{CreateAuditKey})$
  - Compute $(pk_u^a, sk_u^a) := \mathsf{CreateAuditKey}(pp)$.
  - Add the address pair (pk_u^a, sk_u^a) to AUD^{PRO}.
  - Output the address public key pk_u^a
- $\mathcal{Q} = (\mathsf{Deposit}, v, addr_{pk})$
  - Compute $(c, tx_{deposit}) := \mathsf{Deposit}(pp, v, addr_{pk})$
  - Add the deposit transaction $tx_{deposit}$ to L
  - Output $\perp$
- $\mathcal{Q} = (\mathsf{Withdraw}, idx_1, idx_2, , addr_{sk,1}, addr_{sk,2}, v^{pub}, v_1^{new}, v_2^{new}, addr_{pk,1}^{new}, addr_{pk,2}^{new}, (sk_u^a, pk_u^a), pk_{enc,1}^a, pk_{enc,2}^a, pk_{enc,3}^a, ADDR)$
  - Compute rt, the root of a Merkle tree over all coin commitments in L^{PRO}
  - For each $i \in \{1, 2\}$: Let cm_i be the idx_i-th coin commitment in L, tx_i be the deposit/withdraw transaction in L^{PRO} that contains cm_i, c_i be the first coin in $COIN^{PRO}$ with coin commitment cm, $(addr_{pk,i}, addr_{sk,i})$ be the first key pair in $ADDR^{PRO}$ with $addr_{pk,i}$ being c_i's address. Compute $path_i$, the authentication path from cm_i to rt
  - Compute $(c_1^{new}, c_2^{new}, tx_{withdraw}) := \mathsf{Withdraw}(pp, rt, path_1, path_2, addr_{sk,1}, addr_{sk,2}, v^{pub}, v_1^{new}, v_2^{new}, addr_{pk,1}^{new}, addr_{pk,2}^{new}, (sk_u^a, pk_u^a), pk_{enc,1}^a, pk_{enc,2}^a, pk_{enc,3}^a, ADDR)$
  - Verify that $\mathsf{VerifyTransaction}(pp, tx_{withdraw}, L)$ outputs 1.
  - Add the withdraw transaction to L.
  - Output $\perp$.
- $\mathcal{Q} = (\mathsf{Rollup}, N^{rollup})$
  - Look up a queue Q^{cm} containing N^{rollup} commitments waiting for rollup in L^{PRO}. (If no such transaction is found, abort.)
  - Let $c_1, \ldots, c_n$ be the coins in Q^{cm}.

- Compute rt^{old}, the root of a Merkle tree over all rolluped coin commitments in L^{PRO}.
- Compute $path$, the authentication path from commitments to rt^{old}.
- Compute $tx_{rollup} \leftarrow \mathsf{Rollup}(pp, N^{rollup}, Q^{cm}, rt^{old}, path)$.
- Add $c_1, ..., c_n$ to $COIN^{PRO}$
- Output $\perp$.

- $\mathcal{Q} = (\mathsf{Receive}, addr_{pk})$
  - Look up $(addr_{pk}, addr_{sk})$ in $ADDR^{PRO}$. (If no such key pair is found, abort.)
  - Compute $(c_1, ..., c_n) \leftarrow \mathsf{Receive}(pp, (addr_{sk}, addr_{pk}), L^{PRO})$.
  - Add $c_1, ..., c_n$ to $COIN^{PRO}$
  - Output $(cm_1, ..., cm_n)$ the corresponding coin commitments.

- $\mathcal{Q} = (\mathsf{Insert}, tx)$
  - Verify that $\mathsf{VerifyTransaction}(pp, tx, L)$ outputs 1. (Else, abort.)
  - Add the deposit/withdraw transaction tx to L^{PRO}
  - Run Rollup and Reveive for all address $addr_{pk}$ in $ADDR$; this updates the $COIN^{PRO}$ with any coins that might have been sent to honest parties via tx.
  - Output $\perp$.

6.2.1. Ledger indistinguishability

Definition 4. *Let* Π=(Setup, CreateAddress, CreateAuditKey, Deposit, Withdraw, Rollup, VerifyTransaction, Receive, Audit) *be a protocol. We say that* Π *is* L-IND *secure if, for every poly(λ)-size adversary* $\mathcal{A}$ *and sufficiently large* λ, $Adv^{\mathsf{L\text{-}IND}}_{\Pi,A}(\lambda) < negl(\lambda)$, *where* $Adv^{\mathsf{L\text{-}IND}}_{\Pi,A}(\lambda) := 2 \cdot Pr[\mathsf{L\text{-}IND}(\Pi, A, \lambda) = 1] - 1$ *is* $\mathcal{A}$*'s advantage in the* L-IND *experiment.*

We now describe the ledger indistinguishability experiment L-IND. This experiment is an interactive game between an adversary $\mathcal{A}$ and a challenger $\mathcal{C}$.

Setup. At the beginning, $\mathcal{C}$ samples a random bit $b \in (0, 1)$ and public parameters $pp \leftarrow \mathsf{Setup}(1^\lambda)$, and sends pp to $\mathcal{A}$. $\mathcal{C}$ then initializes two oracles $\mathcal{O}_0^{PRO}$ and $\mathcal{O}_1^{PRO}$ using pp.

Main part. Let L_{left} be the current ledger in $\mathcal{O}_b^{PRO}$ and L_{right} be the current ledger in $\mathcal{O}_{1-b}^{PRO}$. $\mathcal{C}$ provides (L_{left}, L_{right}) to $\mathcal{A}$; $\mathcal{A}$ then sends two queries:

$$\mathcal{Q}, \mathcal{Q}' \in \{\mathsf{CreateAddress}, \mathsf{CreateAuditKey}, \mathsf{Deposit}, \mathsf{Withdraw}, \mathsf{Rollup}, \mathsf{Receive}, \mathsf{Insert}\}$$

to $\mathcal{C}$, while $\mathcal{Q}$ and $\mathcal{Q}'$ should be publicly consistent. If query type is Insert, $\mathcal{C}$ forwards $\mathcal{Q}$ to $\mathcal{O}_b^{PRO}$, and $\mathcal{Q}'$ to $\mathcal{O}_{1-b}^{PRO}$. Otherwise, $\mathcal{C}$ first check if $\mathcal{Q}$ and $\mathcal{Q}'$ are publicly consistent and then forwards $\mathcal{Q}$ to $\mathcal{O}_0^{PRO}$ and $\mathcal{Q}'$ to $\mathcal{O}_1^{PRO}$. Let a_0 and a_1 be the two oracle answers, $\mathcal{C}$ then sends (a_b, a_{1-b}) to $\mathcal{A}$.

$\mathcal{A}$ and $\mathcal{C}$ may repeat the **Main part** several times. At the end of the experiment, $\mathcal{A}$ sends $\mathcal{C}$ a guess bit $b' \in (0, 1)$. $\mathcal{C}$ outputs 1 if $b = b'$, or 0 otherwise.

Public consistency Two queries $\mathcal{Q}$ and $\mathcal{Q}'$ are publicly consistent iff $\mathcal{Q}$ and $\mathcal{Q}'$ are the same type. Furthermore, they are well formatted, and their public information is equal.

6.2.2. Transaction non-malleability

Definition 5. *Let* $\Pi=($Setup, CreateAddress, CreateAuditKey, Deposit, Withdraw, Rollup, VerifyTransaction, Receive, Audit*) be a protocol. We say that* Π *is* TR-NM *secure if, for every poly(λ)-size adversary* $\mathcal{A}$ *and sufficiently large* λ, $Adv^{\mathsf{TR\text{-}NM}}_{\Pi,A}(\lambda) < negl(\lambda)$, *where* $Adv^{\mathsf{TR\text{-}NM}}_{\Pi,A}(\lambda) := 2 \cdot Pr[\mathsf{TR\text{-}NM}(\Pi, A, \lambda) = 1] - 1$ *is* $\mathcal{A}$*'s advantage in the* TR-NM *experiment.*

We now describe the transaction non-malleability experiment TR-NM. This experiment is an interactive game between an adversary $\mathcal{A}$ and a challenger $\mathcal{C}$. At the beginning, $\mathcal{C}$ samples $pp \leftarrow \mathsf{Setup}(1^\lambda)$, and sends pp to $\mathcal{C}$. $\mathcal{C}$ then initializes an oracle $\mathcal{O}^{PRO}$using pp. $\mathcal{A}$ may send several queries to $\mathcal{O}^{PRO}$. At the end of the experiment, $\mathcal{A}$ sends a withdraw transaction tx' to $\mathcal{C}$. Let $\mathbb{T}$ be the set of all withdraw transaction generated by $\mathcal{O}^{PRO}$. $\mathcal{C}$ outputs 1 iff there exists a $tx \in \mathbb{T}$ s.t. (1) $tx' \neq tx$; (2) $\mathsf{VerifyTransaction}(pp, tx', L) = 1$; and (3) a serial number revealed in tx' is also revealed in tx.

6.2.3. Balance

Definition 6. *Let* $\Pi=($Setup, CreateAddress, CreateAuditKey, Deposit, Withdraw, Rollup, VerifyTransaction, Receive, Audit*) be a protocol. We say that* Π *is* BAL *secure if, for every poly(λ)-size adversary* $\mathcal{A}$ *and sufficiently large* λ, $Adv^{\mathsf{BAL}}_{\Pi,A}(\lambda) < negl(\lambda)$, *where* $Adv^{\mathsf{BAL}}_{\Pi,A}(\lambda) := 2 \cdot Pr[\mathsf{BAL}(\Pi, A, \lambda) = 1] - 1$ *is* $\mathcal{A}$*'s advantage in the* BAL *experiment.*

We now describe the balance experiment BAL. This experiment is an interactive game between an adversary $\mathcal{A}$ and a challenger $\mathcal{C}$. At the beginning, $\mathcal{C}$ samples $pp \leftarrow \mathsf{Setup}(1^\lambda)$, and sends pp to $\mathcal{A}$. $\mathcal{C}$ then initializes an oracle $\mathcal{O}^{PRO}$using pp. $\mathcal{A}$ may send several queries to $\mathcal{O}^{PRO}$. At the end of the experiment, $\mathcal{A}$ sends $\mathcal{C}$ a set of coins $\mathbb{C}$. $\mathcal{C}$ computes the following quantities.

- $v_{unwithdrawn}$, the total withdrawable coins in $\mathbb{C}$.
- $v_{deposit}$, the total value of all coins deposited by $\mathcal{A}$.
- $v_{ADDR^{PRO}\rightarrow A}$, the total value of payment received by $\mathcal{A}$ from addresses in $ADDR^{PRO}$.
- $v_{A\rightarrow ADDR^{PRO}}$, the total value of payment sent by $\mathcal{A}$ to addresses in $ADDR^{PRO}$.
- $v_{basecoin}$, the total value of public outputs placed by $\mathcal{A}$ on the ledger.

$\mathcal{C}$ outputs 1 iff $v_{unwithdrawn} + v_{basecoin} + v_{A\rightarrow ADDR^{PRO}} > v_{deposit} + v_{ADDR^{PRO}\rightarrow A}$.

6.2.4. Auditability

Definition 7. *Let* $\Pi=($Setup, CreateAddress, CreateAuditKey, Deposit, Withdraw, Rollup, VerifyTransaction, Receive, Audit*) be a protocol. We say that* Π *is* AUD *secure if, for every poly(λ)-size adversary* $\mathcal{A}$ *and sufficiently large* λ, $Adv^{\mathsf{AUD}}_{\Pi,A}(\lambda) < negl(\lambda)$, *where* $Adv^{\mathsf{AUD}}_{\Pi,A}(\lambda) := 2 \cdot Pr[\mathsf{AUD}(\Pi, A, \lambda) = 1] - 1$ *is* $\mathcal{A}$*'s advantage in the* AUD *experiment.*

We now describe the balance experiment AUD. This experiment is an interactive game between an adversary $\mathcal{A}$ and a challenger $\mathcal{C}$. At the beginning, $\mathcal{C}$ samples $pp \leftarrow \mathsf{Setup}(1^\lambda)$, and sends pp to $\mathcal{A}$. $\mathcal{C}$ then initializes an oracle $\mathcal{O}^{PRO}$using pp. $\mathcal{A}$ may send several queries to $\mathcal{O}^{PRO}$. At the end of the experiment, $\mathcal{A}$ sends a withdraw transaction tx to $\mathcal{C}$. $\mathcal{C}$ outputs 1 iff (1) $\mathsf{VerifyTransaction}(pp, tx, L) = 1$; and (2) the outputs of $\mathsf{Audit}(\mathsf{msg}^{\mathsf{a}}_1, \mathsf{msg}^{\mathsf{a}}_2, \mathsf{msg}^{\mathsf{a}}_3, \mathsf{pk}^{\mathsf{a}}_{\mathsf{u}}, \mathsf{sk}^{\mathsf{a}}_{\mathsf{enc},1}, \mathsf{sk}^{\mathsf{a}}_{\mathsf{enc},2}, \mathsf{sk}^{\mathsf{a}}_{\mathsf{enc},3})$ are not equal to the input commitments to tx.

6.3. Proof of Theorem 1

In this section, we will sketch the proof of the Theorem 1. Similar to [1], we also omit the proof of completeness. We then prove the security with three separate proofs.

6.3.1. Ledger indistinguishability.

We prove this property by hybrid experiments from the ledger indistinguishability experiment $\mathsf{L\text{-}IND}$ to a simulation $\mathsf{SIM}^{\mathsf{L\text{-}IND}}$. In the simulation, the adversary $\mathcal{A}$ interacts with a challenger $\mathcal{C}$ as in the experiment, except that all answers are computed independently of the bit $\mathcal{B}$. We then prove that the simulation is indistinguishable from the real experiments.

The simulation $\mathsf{SIM}^{\mathsf{L\text{-}IND}}$ works as follows. The setup stage is similar to the $\mathsf{L\text{-}IND}$ experiment. However, the zk-SNARKs for withdrawing coins and ZK-rollup are initialized with two simulations $\mathsf{SIM}^{zk}_{WITHDRAW}, \mathsf{SIM}^{zk}_{ROLLUP}$. Then, the challenger $\mathcal{C}$ answers different queries as follows.

- $\mathsf{CreateAddress}$. $\mathcal{C}$ behaves as in $\mathsf{L\text{-}IND}$, except that $\mathcal{C}$ replaces a_{pk} in $addr_{pk}$ with a random string. Then, $\mathcal{C}$ stores $addr_{sk}$ in a table and returns $addr_{pk}$ to $\mathcal{A}$.
- $\mathsf{CreateAuditKey}$. The answer is unique to the $\mathsf{L\text{-}IND}$ experiment.
- $\mathsf{Deposit}$. $\mathcal{C}$ behaves as in $\mathsf{L\text{-}IND}$, except that $\mathcal{C}$ computes k as $COMM_r(\tau||\rho)$ where τ is a random string, and $\mathcal{C}$ samples a pk_{enc} and calculates $Ct := E_{enc}(pk_{enc}, r)$ where r is a random string.
- $\mathsf{Withdraw}$. $\mathcal{C}$ computes rt as the accumulation of all the coin commitments after ZK-rollup on L_i. Then, $\mathcal{C}$ samples two uniformly randoms sn_1^{old} and sn_2^{old}. For $i \in \{1, 2\}$, if $addr_{pk,i}^{new}$ is generated by $\mathsf{CreateAddress}$, $\mathcal{C}$ samples a random cm_i^{new} and a random pk_{enc} and calculates $Ct_i^{new} := E_{enc}(pk_{enc}, r)$ where r is a random string. For $j \in \{1, 2, 3\}$, $\mathcal{C}$ samples a random k_j^a and calculates $msg_j^a = SEC.Enc_{k_j^a}(r)$ where r is a random string. Otherwise, calculate cm_i^{new} and Ct_i^{new} as in the $\mathsf{Withdraw}$ algorithm. Let h_{sig} be a random string and compute all remain value as in $\mathsf{Withdraw}$ algorithm. $\mathcal{C}$ computes the proof π_{WITHDRAW} from the simulation $\mathsf{SIM}^{zk}_{WITHDRAW}$.
- Rollup. $\mathcal{C}$ behaves as in $\mathsf{L\text{-}IND}$, except that $\mathcal{C}$ computes the proof π_{ROLLUP} from the simulation $\mathsf{SIM}^{zk}_{ROLLUP}$.
- $\mathsf{Receive}$. The answer is unique to the $\mathsf{L\text{-}IND}$ experiment.
- Insert. The answer is unique to the $\mathsf{L\text{-}IND}$ experiment.

In each case, the answer to $\mathcal{A}$ is independent from the bit $\mathcal{B}$. When $\mathcal{A}$ guesses the bit $\mathcal{B}$, $\mathcal{A}$ can only sample a random bit b', i.e., $\mathcal{A}$'s advantage is 0. Next, we will prove that $\mathsf{SIM}^{\mathsf{L\text{-}IND}}$ is indistinguishable from $\mathsf{L\text{-}IND}$.

Sketch of Proof: We now describe a sequence of hybrid experiments

$$(\mathsf{L\text{-}IND}, \mathsf{SIM}^{\mathsf{L\text{-}IND}_1}, \mathsf{SIM}^{\mathsf{L\text{-}IND}_2}, \mathsf{SIM}^{\mathsf{L\text{-}IND}_3}, \mathsf{SIM}^{\mathsf{L\text{-}IND}})$$

Let $q_{\mathbf{CA}}$ be the number of $\mathsf{CreateAddress}$ queries issued by $\mathcal{A}$. Let $q_{\mathbf{S}}$ be the number of $\mathsf{Withdraw}$ queries issued by $\mathcal{A}$. Let $q_{\mathbf{D}}$ be the number of $\mathsf{Deposit}$ queries issued by $\mathcal{A}$. For each intermediate experiments, we modify the experiment and show that it is distinguishable from the previous experiment.

- $\mathsf{SIM}^{\mathsf{L\text{-}IND}_1}$: In experiment $\mathsf{SIM}^{\mathsf{L\text{-}IND}_1}$, we simulate the zk-SNARK. For each withdraw transaction, $\mathcal{C}$ computes the proof π_{WITHDRAW} from a simulation $\mathsf{SIM}^{zk}_{WITHDRAW}$. For each ZK-rollup transaction, $\mathcal{C}$ computes the proof π_{ROLLUP} from a simulation $\mathsf{SIM}^{zk}_{ROLLUP}$. Since zk-SNARK is perfect zero knowledge, the simulation proof π_{WITHDRAW} and π_{ROLLUP} should be indistinguishable from a real proof. Hence $Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}_1}} = 0$.

- $\mathsf{SIM}^{\mathsf{L\text{-}IND}_2}$: The experiment $\mathsf{SIM}^{\mathsf{L\text{-}IND}_2}$ modifies experiment $\mathsf{SIM}^{\mathsf{L\text{-}IND}_1}$ by replacing ciphertext in withdraw transactions with an encryption of a random string. More precisely, we modify $\mathsf{SIM}^{\mathsf{L\text{-}IND}_2}$ so that:

  - each time $\mathcal{A}$ issues a $\mathsf{Deposit}$ query, $\mathcal{C}$ samples a pk_{enc} and calculates $Ct := E_{enc}(pk_{enc}, r)$ where r is a random string.,
  - each time $\mathcal{A}$ issues a $\mathsf{Withdraw}$ query, for $i \in \{1, 2\}$, if $addr^{new}_{pk,i}$ is generated by $\mathsf{CreateAddress}$, $\mathcal{C}$ samples a random pk_{enc} and calculates $Ct^{new}_i := E_{enc}(pk_{enc}, r)$ where r is a random string. For $j \in \{1, 2, 3\}$, $\mathcal{C}$ samples a random k^a_j and calculates $msg^a_j = SEC.Enc_{k^a_j}(r)$ where r is a random string.

  By Lemma 1, we claim that $|Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}_2}} - Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}_1}}| \leq 2(q_{\mathbf{D}} + 5q_{\mathbf{S}})Adv^{\mathsf{Enc}}$.

- $\mathsf{SIM}^{\mathsf{L\text{-}IND}_3}$: The experiment $\mathsf{SIM}^{\mathsf{L\text{-}IND}_3}$ modifies $\mathsf{SIM}^{\mathsf{L\text{-}IND}_2}$ by replacing all PRF results with random values. More precisely, we modify $\mathsf{SIM}^{\mathsf{L\text{-}IND}_2}$ so that :

  - each time $\mathcal{A}$ issues a $\mathsf{CreateAddress}$ query, the value a_{pk} in $addr_{pk}$ is substituted with a random string of the same length; and
  - each time $\mathcal{A}$ issues a $\mathsf{Withdraw}$ query, the serial number sn^{old} and the signature h are substituted with random strings of the same length.

  By Lemma 2, we claim that $|Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}_3}} - Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}_2}}| \leq q_{\mathbf{CA}}Adv^{\mathsf{PRF}}$.

- $\mathsf{SIM}^{\mathsf{L\text{-}IND}}$: We already described the experiment $\mathsf{SIM}^{\mathsf{L\text{-}IND}}$ above. More precisely, we modify $\mathsf{SIM}^{\mathsf{L\text{-}IND}_2}$ so that each time $\mathcal{A}$ issues a $\mathsf{Deposit}$ query, the commitment cm in $tx_{deposit}$ is substituted with a commitment to a random input. If $\mathcal{A}$ issues a $\mathsf{Withdraw}$ query, for $i \in \{1, 2\}$, if $addr^{new}_{pk,i}$ is generated by $\mathsf{CreateAddress}$, $\mathcal{C}$ samples a random cm^{new}_i. By Lemma 3, we claim that $|Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}}} - Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}_3}}| \leq (q_{\mathbf{D}} + 4q_{\mathbf{S}})Adv^{\mathsf{COMM}}$.

By summing over $\mathcal{A}$'s advantages in the hybrid experiments, we can bound $\mathcal{A}$'s advantage in $\mathsf{L\text{-}IND}$ by $Adv^{\mathsf{L\text{-}IND}}_{\Pi,A}(\lambda) \leq 4(q_{\mathbf{D}} + q_{\mathbf{S}})Adv^{\mathsf{Enc}} + 2(q_{\mathbf{D}} + 5q_{\mathbf{S}})Adv^{\mathsf{Enc}} + (q_{\mathbf{D}} + 4q_{\mathbf{S}})Adv^{\mathsf{COMM}}$, which is negligible in λ.

Lemma 1. *Let Adv^{Enc} be $\mathcal{A}$'s advantages in the* IND-CCA *and* IK-CCA *experiments against the encryption scheme Enc. Since the Ecies scheme is an instance of Enc, we use Adv^{Enc} to denote $\mathcal{A}$'s advantages in the* IND-CCA *and* IK-CCA *experiments against the encryption scheme Ecies. Then after $q_{\mathbf{D}}$* Deposit *queries and $q_{\mathbf{S}}$* Withdraw *queries, $|Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}_2}} - Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}_1}}| \leq 4(q_{\mathbf{D}} + q_{\mathbf{S}})Adv^{\mathsf{Enc}}$.*

Proof sketch. Let H be an intermediate simulation between $\mathsf{SIM}^{\mathsf{L\text{-}IND}_1}$ and $\mathsf{SIM}^{\mathsf{L\text{-}IND}_2}$ in which H modifies $\mathsf{SIM}^{\mathsf{L\text{-}IND}_1}$ by replacing the encrypt key with a new sampled key from the key generation algorithm. We first discuss H and $\mathsf{SIM}^{\mathsf{L\text{-}IND}_1}$. When $\mathcal{A}$ queries $\mathsf{CreateAddress}$, $\mathcal{C}$ queries the $\mathsf{IK\text{-}CCA}$ challenger to obtain $(pk_{enc,0}, pk_{enc,1})$ and return $pk_{enc} := pk_{enc,0}$ to $\mathcal{A}$. When $\mathcal{A}$ issues a $\mathsf{Deposit}$

query or a Withdraw query, $\mathcal{C}$ queries the IK-CCA challenger over the plain text m and receive $Ct^* := E_{enc}(pk_{enc,b}, m)$ where $\mathcal{B}$ is chosen by the IK-CCA challenger. $\mathcal{C}$ then replaces Ct with Ct^* and adds the result $tx_{deposit}$ to the ledger. $\mathcal{A}$ outputs a bit b', which is our answer to the IK-CCA experiment. If the maximum advantage of the IK-CCA experiment is Adv^{Enc}, by using a hybrid game, we say that $|Adv^{\mathsf{SIM}^{\mathsf{H}}} - Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}_1}}| \leq (q_{\mathbf{D}} + 5q_{\mathbf{S}})Adv^{\mathsf{Enc}}$.

We made a similar argument for H and $\mathsf{SIM}^{\mathsf{L\text{-}IND}_2}$. Overall, the advantage is $|Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}_2}} - Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}_1}}| \leq 2(q_{\mathbf{D}} + 5q_{\mathbf{S}})Adv^{\mathsf{Enc}}$.

Lemma 2. *Let Adv^{PRF} be $\mathcal{A}$'s advantages in distinguishing PRF from a true random function. Then after $q_{\mathbf{CA}}$ CreateAddress queries, $|Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}_3}} - Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}_2}}| \leq q_{\mathbf{CA}}Adv^{\mathsf{PRF}}$.*

Proof sketch. As mentioned in Section 5.1, all pseudorandom functions $PRF^{addr}_{a_{sk}}$, $PRF^{sn}_{a_{sk}}$, and $PRF^{pk}_{a_{sk}}$ are constructed from $PRF_{a_{sk}}$. Let $\mathcal{O}$ be the oracle implementing $PRF_{a_{sk}}$ or a true random function. Let a_{sk} be the random seed generated by the oracle from the PRF experiment in answering the first CreateAddress query. If $\mathcal{O}$ implements the $PRF_{a_{sk}}$, the experiment distribution is identical to $\mathsf{SIM}^{\mathsf{L\text{-}IND}_2}$. By using a hybrid game, we say that $|Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}_3}} - Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}_2}}| \leq q_{\mathbf{CA}}Adv^{\mathsf{PRF}}$.

Lemma 3. *Let Adv^{COMM} be $\mathcal{A}$'s advantages against the hiding property of COMM. Then after $q_{\mathbf{D}}$ Deposit queries and $q_{\mathbf{S}}$ Withdraw queries, $|Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}}} - Adv^{\mathsf{SIM}^{\mathsf{L\text{-}IND}_3}}| \leq (q_{\mathbf{D}} + 4q_{\mathbf{S}})Adv^{\mathsf{COMM}}$.*

Proof sketch. We make a similar argument as in Lemma 2. We replace internal commitments in k in Deposit and Withdraw with a random value. By using a hybrid game, the advantage is bounded by $(q_{\mathbf{D}} + 2q_{\mathbf{S}})Adv^{\mathsf{COMM}}$. We then replace the coin commitment in Withdraw with a random value, the overall advantage is $(q_{\mathbf{D}} + 4q_{\mathbf{S}})Adv^{\mathsf{COMM}}$.

6.3.2. Transaction non-malleability.

Define $\epsilon := Adv^{\mathsf{TR\text{-}NM}}_{\Pi,A}(\lambda)$. Let $\mathbb{T}$ be the set of withdraw transactions generated by $\mathcal{O}^{PRO}$ in response to Withdraw queries. Set $h'_{sig} := CRH(pk'_{sig})$ corresponding to tx'. Let pk_{sig} be the corresponding public key in tx and set $h_{sig} := CRH(pk_{sig})$. Let $\mathcal{Q}_{\mathbf{CA}} = \{a_{sk,1}, ..., a_{sk,q_{\mathbf{CA}}}\}$ be the set of internal address keys created by $\mathcal{C}$ in response to $\mathcal{A}$'s CreateAddress queries. Let $\mathcal{Q}_{\mathbf{S}} = \{pk_{sig,1}, ..., pk_{sig,q_{\mathbf{S}}}\}$ be the set of signature public keys created by $\mathcal{C}$ in response to $\mathcal{A}$'s Withdraw queries. Then, we decompose the event in which $\mathcal{A}$ wins into the following four disjoint events.

- EVENT_{sig} : $\mathcal{A}$ wins the TR-NM experiment, and there is $pk''_{sig} \in \mathcal{Q}_{\mathbf{S}}$ such that $pk'_{sig} = pk''_{sig}$.
- EVENT_{col} : $\mathcal{A}$ wins, and the above event does not occur, and there is $pk''_{sig} \in \mathcal{Q}_{\mathbf{S}}$ such that $h'_{sig} = CRH(pk''_{sig})$.
- EVENT_{mac} : $\mathcal{A}$ wins, and above two events do not occur, and $h' = PRF^{pk}_{a}(i|h_{sig})$ for some $i \in \{1, 2\}$ and $a \in \mathcal{Q}_{\mathbf{CA}}$.
- EVENT_{key} : $\mathcal{A}$ wins, and above three events do not occur, and $h' \neq PRF^{pk}_{a}(i|h_{sig})$ for all $i \in \{1, 2\}$ and $a \in \mathcal{Q}_{\mathbf{CA}}$.

Clearly, $\epsilon = Pr[\mathsf{EVENT}_{sig}] + Pr[\mathsf{EVENT}_{col}] + Pr[\mathsf{EVENT}_{mac}] + Pr[\mathsf{EVENT}_{key}]$. Then, we bound the probability of each event and show that they are all negligible to λ.

Bound the probability of EVENT_{sig}: Define $\epsilon_1 := Pr[\mathsf{EVENT}_{sig}]$. We proof the statement that ϵ_1 is negligible in λ by contradiction. More precisely, if ϵ_1 is not negligible, $\mathcal{A}$ can forge the signature with more than negligible probability, which breaks the SUF-1CMA security.

Let σ' be the signature in tx', and σ'' be the signature in the first withdraw transaction in $tx'' \in \mathbb{T}$ that contains pk''_{sig}. Let m' be everything in tx' other than σ'. Let m'' be everything in tx'' other than σ''. Observe that whenever $tx' \neq tx''$ we also have $(m', \sigma') \neq (m'', \sigma'')$. We first show that $tx' = tx''$ with negligible probability by contradiction. Since, by the definition of TR-NM, tx' and tx share the same serial number. Suppose $tx' = tx''$ then tx and tx'' also share the same serial number, which is bound by the negligible probability that $\mathbb{T}$ contains two transactions that share the same serial number.

Next, we describe an algorithm $\mathcal{B}$, which uses $\mathcal{A}$ as a subroutine, that wins the SUF-1CMA game against Sig with $\epsilon_1/q_{\mathbf{P}}$.

1. $\mathcal{B}$ chooses a random $i \in \{1, 2 \ldots, q_{\mathbf{S}}\}$
2. $\mathcal{B}$ conducts the TR-NM with $\mathcal{A}$. When $\mathcal{A}$ issues the i-th Withdraw query, $\mathcal{B}$ substitutes pk''_{sig} in the resulting transactions tx''. $\mathcal{B}$ then queries the SUF-1CMA challenger and obtains the signature σ'' for the message m'' and substitutes σ'' in tx''.
3. When $\mathcal{A}$ outputs tx', $\mathcal{B}$ looks into tx' and finds m' and σ'.
4. If $pk''_{sig} \neq pk'_{sig}$, $\mathcal{B}$ aborts; otherwise, $\mathcal{B}$ outputs m' and σ' as a forgery for Sig.

Because Sig is SUF-1CMA, ϵ_1 must be negligible in λ.

Bound the probability of EVENT_{col}: Define $\epsilon_2 := Pr[\mathsf{EVENT}_{col}]$. When EVENT_{col} occurs, $\mathcal{A}$ finds a collision $CRH(pk'_{sig}) = CRH(pk''_{sig})$. Because CRH is collision resistant, ϵ_2 must be negligible in λ.

Bound the probability of EVENT_{mac}: Define $\epsilon_3 := Pr[\mathsf{EVENT}_{mac}]$. We state that if EVENT_{mac} occurs, the adversary $\mathcal{A}$ can distinguish the pseudorandom function (PRF) from a truly random function. Hence, ϵ_3 must be negligible in the security parameter λ.

Bound the probability of EVENT_{key}: Define $\epsilon_4 := Pr[\mathsf{EVENT}_{key}]$. If EVENT_{key} occurs, there exists an algorithm $\mathcal{B}$ s.t. $\mathcal{B}$ finds collisions for PRF^{sn}. Therefore, ϵ_4 must be negligible in λ.

6.3.3. Balance.

To withdraw more coins than a user owns, $\mathcal{A}$ may insert a transaction on the ledger. We now modify the experiment in a way that does not affect $\mathcal{A}$'s view. For each zk-SNARK instance $x := ([(rt, sn, h)], v^{pub}, cm_1^{new}, cm_2^{new}, h_{sig}, pk_u^a, [pk_{enc}^a], [msg^a])$ in a withdraw transaction, $\mathcal{C}$ computes a witness $a := ([(path, c, addr_{sk})], c_1^{new}, c_2^{new}, [cm^a], sk_u^a)$. $\mathcal{C}$ may do so with a knowledge extractor. Afterwards, $\mathcal{C}$ obtains an *augmented ledger* $(L, \vec{a})$ where $\vec{a}$ is a list of witness a. Note that $(L, \vec{a})$ is a list of matched pairs $(tx_{withdraw}, a)$ where $tx_{withdraw}$ is a withdraw transaction and a is the corresponding witness. Define $\epsilon := Adv_{\Pi,A}^{\mathsf{BAL}}(\lambda)$. We then define the balance property respected to the modified BAL experiment. We say an augmented ledger balanced if the following holds:

1. Each $(tx_{withdraw}, a)$ in $(L, \vec{a})$ contains openings of two valid coin commitments cm_1 and cm_2, and each cm is a output coin commitment of a deposit or withdraw transaction preceding $tx_{withdraw}$ on L.
2. No two $(tx_{withdraw}, a)$ and $(tx'_{withdraw}, a')$ in $(L, \vec{a})$ contain openings of the same coin commitment.
3. Each $(tx_{withdraw}, a)$ in $(L, \vec{a})$ contains opening of cm_1 cm_2 cm_1^{new} cm_2^{new} to value v_1 v_2 v_1^{new} v_2^{new}, and $v_1 + v_2 = v_1^{new} + v_2^{new} + v^{pub}$.

4. For each $(tx_{withdraw}, a)$ in $(L, \vec{a})$, and for each $i \in \{1, 2\}$:

   (a) If cm_i is the output of a deposit transaction $tx_{deposit}$ on L, then the value v in $tx_{deposit}$ is equal to v_i.

   (b) If cm_i is the output of a withdraw transaction $tx'_{withdraw}$ on L, then its witness a' contains an opening of cm_i to a value v' that is equal to v_i

5. For each $(tx_{withdraw}, a)$ in $(L, \vec{a})$, where $tx_{withdraw}$ is inserted by $\mathcal{A}$, if any cm is the output of a previous transaction tx', the public address in tx' is not in $ADDR^{PRO}$. Recall that $ADDR^{PRO}$ is the set of address pairs created by $\mathcal{A}$'s CreateAddress queries.

We then prove that $\mathcal{A}$ cannot violate each case with more than negligible probability.

$\mathcal{A}$ violates Condition 1: By the construction of $\mathcal{O}^{PRO}$, $\mathcal{A}$ cannot violate the condition.

$\mathcal{A}$ violates Condition 2: If $\mathcal{A}$ violates Condition 2, L contains two withdraw transactions $tx_{withdraw}$ and $tx'_{withdraw}$ with the same cm. Since both transactions are valid, they must contain different serial numbers, namely $sn = sn'$. However, if both transactions withdraw cm but produces different serial numbers, then the corresponding witness a, a' contains different openings of cm. This violates the binding property of the commitment scheme $COMM$.

$\mathcal{A}$ violates Condition 3: By the construction of the NP statement $WITHDRAW$, this must hold. Otherwise, the zk-SNARK is violated.

$\mathcal{A}$ violates Condition 4: If $\mathcal{A}$ violates Condition 4, L contains:

1. a deposit transaction $tx_{deposit}$ and a withdraw transaction $tx_{withdraw}$, or

2. two withdraw transactions $tx_{withdraw}$ and $tx'_{withdraw}$

s.t. both transactions have the same commitment cm but open cm to different values. This violates the binding property of the commitment scheme $COMM$.

$\mathcal{A}$ violates Condition 5: If $\mathcal{A}$ violates Condition 5, L contains an inserted withdraw transaction $tx_{withdraw}$ s.t. $tx_{withdraw}$ withdraws a coin deposited by a previous deposit transaction $tx_{deposit}$. Notably, $tx_{deposit}$'s public address $addr_{pk} = (a_{pk}, pk_{enc})$ lies in $ADDR$, and the witness associated to $tx_{deposit}$ contains a_{sk} s.t. $a_{pk} = PRF^{addr}_{a_{sk}}(0)$. One can construct a new adversary $\mathcal{B}$ that, by using $\mathcal{A}$ as a subroutine, distinguishes PRF from a random function.

6.3.4. Auditability.

If the auditors fail to recover the commitments, it implies:

1. any decryption of the messages is wrong; or

2. the secret sharing recovery result is wrong; or

3. π_{WITHDRAW} in tx is valid while $[cm^a] \neq Share^{(t,n)}([cm])$.

$\mathcal{A}$ violates Condition 1: If Condition 1 exists, it implies that exists i s.t. for i-th auditor $SEC.Enc_{k_i^a}(cm_i^a) = msg_i^a$ while $SEC.Dec_{k_i^a}(msg_i^a) \neq cm_i^a$, which compromises the security of $ECIES$. Therefore, the possibility of Condition 1 must be negligible in λ.

$\mathcal{A}$ violates Condition 2: If Condition 2 exists, there must be the case that $[cm_1^a, cm_2^a, \ldots, cm_n^a] = Share^{(t,n)}([cm])$ but $[cm] \neq Share^{(t,n)}([cm_1^a, cm_2^a, \ldots, cm_n^a])$. Since the secret sharing schema SS in the protocol is PER-SS security, Condition 2 cannot exist. Therefore, the possibility of Condition 2 must be negligible in λ.

$\mathcal{A}$ **violates Condition 3**: The construction of the NP statement WITHDRAW ensures that this condition must not exist. Otherwise, it would imply a violation of the zk-SNARK.

The protocol ensures auditability through a structured process involving a set of auditors $\mathbb{A}$, a predefined threshold t, and encrypted transaction data. An auditor A_i submits an audit request for a transaction tx. If the threshold is met, the encrypted audit data in tx is decrypted using the secret shares. The threshold t is configurable based on regulatory and system requirements, ensuring a balance between privacy protection and compliance enforcement.

Each auditor operates under constraints to prevent misuse. Auditors have *read-only access*, meaning they can only view past encrypted transaction data but cannot modify, reverse, or censor transactions. Transaction validity is maintained through *zero-knowledge proofs*, ensuring that sender and receiver details remain hidden unless decryption is authorized. Furthermore, access control is enforced by *threshold encryption*, where no single auditor can unilaterally decrypt transaction data, preventing unauthorized disclosure. The protocol satisfies three key security properties:

- **Auditability Soundness**: If an auditor follows the protocol correctly, they can retrieve audit information when the threshold t is met.
- **Privacy Preservation**: No single auditor can unilaterally access transaction data, as decryption requires cooperation from t auditors.
- **Adversarial Resilience**: Even if a majority of auditors become adversarial, user funds remain secure since auditors can only view historical transaction data but cannot modify or steal assets.

These guarantees ensure that the protocol maintains auditability while preserving privacy and preventing malicious behavior.

6.4. Attack Scenarios and Defense Measures

To strengthen our security analysis, we consider various possible attack scenarios and discuss the corresponding defense mechanisms.

6.4.1. Double-Spending Attacks

An adversary may attempt to spend the same coin multiple times before the transaction is confirmed in the ledger. Our protocol mitigates this risk through:

- The inclusion of unique serial numbers for each coin ensures that a coin cannot be spent more than once.
- Cryptographic proofs, such as zero-knowledge proofs, that validate the authenticity and uniqueness of a transaction.
- Immediate invalidation of coins spent within the ledger state.

6.4.2. Front-Running Attacks

In decentralized settings, adversaries may attempt to perform front-run transactions by manipulating the transaction order.

- The use of commit-reveal schemes ensures that transaction details remain hidden until they are finalized.
- Encryption mechanisms prevent unauthorized parties from inspecting transaction contents before execution.

6.4.3. Ledger Manipulation Attacks

An attacker could attempt to manipulate the ledger to introduce fraudulent transactions.

- The Merkle tree structure ensures integrity, as each transaction must be cryptographically verified before inclusion.
- Digital signatures and cryptographic commitments make tampering with ledger entries computationally infeasible.

6.4.4. Auditability

To ensure compliance and prevent unauthorized activities, our protocol provides robust auditability features.

- Auditors can verify the correctness of the transaction through the Audit function without revealing sensitive user information.
- Users can generate audit keys that allow trusted third parties to inspect their transactions while maintaining privacy.

6.4.5. Resilience Against an Adversarial Auditor Majority

If a majority of auditors become adversarial, the protocol remains secure because auditors can only view past transaction history and cannot alter transactions, spend user funds, or disrupt the integrity of the protocol. Since auditing is a read-only process, adversarial auditors may attempt to compromise privacy but cannot commit fraud or manipulate user balances.

To mitigate the risk of an adversarial majority, the protocol incorporates several safeguards:

- **Decentralized Auditor Selection:** Prevents collusion by allowing governance-based selection of auditors.
- **Threshold Encryption:** Ensures that no single auditor can access transaction details without approval from a predefined number of independent auditors.
- **Revocation Mechanisms:** Enables governance or stakeholders to replace compromised auditors if malicious behavior is detected.

These enhancements to the security analysis strengthen the resilience of the protocol against real-world attacks and align with formal security guarantees established in the main theorem.

7. Implementation and Experiments

We implemented our protocol on the Ethereum blockchain and evaluated its performance across various system environments, as described in Table 3. The protocol employs the Groth16 proof system and is implemented using ZoKrates [45], a toolbox for zk-SNARKs on Ethereum. We developed smart contracts using the Solidity programming language. The system operates through three main phases: deposit, withdraw, and ZK-rollup. During each phase, the protocol submits a transaction to the blockchain and executes a corresponding smart contract. The withdraw and deposit phases also involve computing SNARKs for withdrawing coins and performing ZK-rollup operations.

It is worth noting that the experiments presented in this work were conducted on the Ethereum blockchain, which is widely used and significant in terms of the data collected. Moreover, many

blockchains, such as Binance Smart Chain (BSC), Polygon, Avalanche, and Fantom, are based on the Ethereum Virtual Machine (EVM) and offer better throughput than Ethereum. While these blockchains may have different configurations, their common EVM compatibility suggests that the proposed protocol could also perform well on these platforms, offering valuable insights into its scalability and efficiency in diverse blockchain environments. A similar approach is adopted in [46].

Machine	OS	CPU	RAM
$\text{Computer}_1(\text{C}_1)$	Ubuntu	Intel(R) Xeon(R) E-2234 CPU @ 3.6 GHz, 8 cores	16GB
$\text{Computer}_2(\text{C}_2)$	macOS	Intel(R) Core(R) i7 CPU @ 2.6 GHz, 6 cores	16GB

Table 3: System specifications of the different testing environments.

7.1. Deposit

During the deposit experiment, a user submits a deposit transaction ($tx_{deposit}$) to the blockchain. The blockchain then locked the token and synced the transaction to the destination chain. The results of the experiment are shown in Table 4.

	Transaction size(byte)	Gas amount (Ethereum)
deposit	484	208,506

Table 4: Deposit experiment results.

7.2. Withdraw

During the withdraw experiment, a user computes a witness and generates the ZK-snark proof as described in Section 5.2. We evaluated the running time (in ms) of each stage during the withdraw phase. σ^2 represents the variance of the running time. We evaluated different withdraw types denoted as $n * m$ where n is the number of input coins and m counts the output coins. The results are shown in Table 5.

7.3. ZK-rollup

During the ZK-rollup experiment, we evaluated the performance of batching multiple transactions into a single proof. The Rollup requires the number of input commitments in the power of 2; therefore, we evaluated the protocol with 2, 4, 8, and 16 commitments. For each type of ZK-rollup, we evaluated the transaction forty times and averaged the measured time. σ^2 represents the variance. The results, including gas consumption and time taken for different batch sizes, are presented in Table 6.

7.4. Discussion

We evaluated the performance and gas consumption of our protocol. The generation of zero-knowledge proofs is the most time-consuming process, taking up to 30 seconds when executing ZK-rollup with 16 commitments on Computer_2. Despite this, the performance is practically acceptable, considering other privacy-preserving payment protocols take significantly longer, e.g., 75 seconds for Zcash and 120 seconds for Monero. A one-to-one private transfer (1×1) consumes an average

Type	ZK-snark	$C_1(ms)[\sigma^2])$	$C_2(ms)[\sigma^2)]$	TX size	Gas
$1*0$	compute-witness	238 [25]	317 [57]	1284	527105
	generate-proof	5140 [145]	5652 [1081]		
	verify	5 [0.25]	6 [0.19]		
$1*1$	compute-witness	292 [25]	322 [59]	1636	618247
	generate-proof	5195 [29]	5757 [215]		
	verify	5 [0]	5 [0.25]		
$1*2$	compute-witness	298 [25]	338 [64]	1988	713191
	generate-proof	5284 [118]	5879 [880]		
	verify	5 [0.2]	6 [0.25]		
$2*0$	compute-witness	562 [6]	627 [52]	1508	629992
	generate-proof	9779 [46]	10900 [2582]		
	verify	5 [0.11]	6 [0.18]		
$2*1$	compute-witness	566 [36]	629 [54]	1860	716400
	generate-proof	9892 [35.06]	10983 [2015]		
	verify	6 [0.23]	6 [0.22]		
$2*2$	compute-witness	571 [30]	639 [32]	2212	811270
	generate-proof	9975 [35]	11119 [4047]		
	verify	6 [0.25]	6 [0.23]		

Table 5: Withdraw experiment results.

of 643,884 gas ($618247 + 410186/16$) when using ZK-rollup with 16 commitments. In comparison, a private transfer in Azeroth [33], an auditable privacy-preserving protocol, consumes 1,555,957 gas. For a 2×2 transaction, the average one-to-one private transfer can be further optimized to 413,146 gas ($811270/2 + 410186/16$).

The audit process in our protocol is triggered only upon request, and in practice, it would typically be performed by trusted entities, such as law enforcement or auditors. As such, the efficiency of the audit process does not directly impact the day-to-day operations, such as deposits, withdrawals, and rollups, which remain unaffected. Furthermore, the audit process is off-chain and primarily involves operations like ECIES encryption and secret sharing, both of which are highly efficient in terms of computational overhead [47]. Thus, while the audit process may involve multiple parties and data verification, its impact on the overall system performance is minimal. However, we acknowledge the importance of considering scalability in large transaction environments and propose future work to explore optimizations for scaling the audit process, especially when handling a large number of audit requests concurrently.

8. Conclusion

In this paper, we presented an auditable confidentiality protocol for blockchain transactions, addressing the significant privacy and security concerns inherent in blockchain technology. Our protocol leverages zero-knowledge proofs and ZK-rollup techniques to provide privacy-preserving cross-chain transactions while ensuring auditability. The implementation on the Ethereum blockchain demonstrated the feasibility and efficiency of our approach.

We evaluated the performance of our protocol in various system environments, highlighting its practical applicability. The results indicate that our protocol achieves a balance between privacy, efficiency, and auditability, making it a viable solution for secure blockchain transactions. The

Type	ZK-snark	$C_1(ms)[\sigma^2]$	$C_2(ms)[\sigma^2]$	TX size	Gas
2	compute-witness	595 [117]	755 [62]	356	327014
	generate-proof	6360 [31]	7604 [2649]		
	verify	3 [0.18]	5 [0.03]		
4	compute-witness	601 [131]	988 [164]	356	331636
	generate-proof	6545 [329]	8314 [276]		
	verify	3 [0.20]	5 [0.08]		
8	compute-witness	1142 [103]	1876 [143]	356	340805
	generate-proof	11057 [91]	14396 [9758]		
	verify	4 [0.20]	5 [0.17]		
16	compute-witness	2223 [140]	3670 [1282]	356	410186
	generate-proof	20700 [1024]	27023 [8522]		
	verify	4 [0.23]	5 [0.09]		

Table 6: Zk-rollup experiment results.

comparison with other privacy-preserving protocols, such as Zcash and Monero, shows that our approach offers competitive performance with lower gas consumption.

Future work includes optimizing the zero-knowledge proof generation process to reduce computational overhead further and extending the protocol to support additional blockchain networks. Additionally, we plan to explore the integration of advanced cryptographic techniques to enhance the protocol's security and scalability.

Overall, our protocol contributes to the advancement of privacy-preserving technologies in the blockchain space, providing a robust solution for secure and auditable transactions across multiple blockchain networks.

References

[1] E. Ben Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, M. Virza, Zerocash: Decentralized anonymous payments from bitcoin, in: 2014 IEEE Symposium on Security and Privacy, 2014, pp. 459–474. doi:10.1109/SP.2014.36.

[2] D. Hopwood, S. Bowe, T. Hornby, N. Wilcox, Zcash protocol specification, GitHub: San Francisco, CA, USA (2016) 1.

[3] S. Noether, A. Mackenzie, Ring confidential transactions, Ledger 1 (2016) 1–18. URL https://ledgerjournal.org/ojs/index.php/ledger/article/view/34

[4] Donate now — zechub.wiki, https://zechub.wiki/guides/Avalanche-RedBridge, [Accessed 27-02-2025].

[5] S. Nakamoto, Bitcoin: A peer-to-peer electronic cash system, Decentralized Business Review (2008) 21260.

[6] G. Wood, et al., Ethereum: A secure decentralised generalised transaction ledger, Ethereum project yellow paper 151 (2014) (2014) 1–32.

[7] S. Goldwasser, S. Micali, C. Rackoff, The knowledge complexity of interactive proof-systems, in: Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing, STOC

'85, Association for Computing Machinery, New York, NY, USA, 1985, p. 291–304. doi:10.1145/22145.22178.
URL https://doi.org/10.1145/22145.22178

[8] M. Blum, P. Feldman, S. Micali, Non-interactive zero-knowledge and its applications, in: Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing, STOC '88, Association for Computing Machinery, New York, NY, USA, 1988, p. 103–112. doi:10.1145/62212.62222.
URL https://doi.org/10.1145/62212.62222

[9] J. Groth, Short pairing-based non-interactive zero-knowledge arguments, in: M. Abe (Ed.), Advances in Cryptology - ASIACRYPT 2010, Springer Berlin Heidelberg, Berlin, Heidelberg, 2010, pp. 321–340.

[10] N. Bitansky, A. Chiesa, Y. Ishai, O. Paneth, R. Ostrovsky, Succinct non-interactive arguments via linear interactive proofs, in: Theory of Cryptography Conference, Springer, 2013, pp. 315–333.

[11] J. Groth, On the size of pairing-based non-interactive arguments, in: M. Fischlin, J.-S. Coron (Eds.), Advances in Cryptology – EUROCRYPT 2016, Springer Berlin Heidelberg, Berlin, Heidelberg, 2016, pp. 305–326.

[12] J. Groth, M. Maller, Snarky signatures: Minimal signatures of knowledge from simulation-extractable snarks, in: Annual International Cryptology Conference, Springer, 2017, pp. 581–612.

[13] S. Bowe, A. Chiesa, M. Green, I. Miers, P. Mishra, H. Wu, Zexe: Enabling decentralized private computation, in: 2020 IEEE Symposium on Security and Privacy (SP), IEEE, 2020, pp. 947–964.

[14] S. Steffen, B. Bichsel, M. Gersbach, N. Melchior, P. Tsankov, M. Vechev, zkay: Specifying and enforcing data privacy in smart contracts, in: Proceedings of the 2019 ACM SIGSAC conference on computer and communications security, 2019, pp. 1759–1776.

[15] S. Steffen, B. Bichsel, R. Baumgartner, M. Vechev, Zeestar: Private smart contracts by homomorphic encryption and zero-knowledge proofs, in: 2022 IEEE Symposium on Security and Privacy (SP), 2022, pp. 179–197. doi:10.1109/SP46214.2022.9833732.

[16] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, G. Maxwell, Bulletproofs: Short proofs for confidential transactions and more, in: 2018 IEEE Symposium on Security and Privacy (SP), 2018, pp. 315–334. doi:10.1109/SP.2018.00020.

[17] B. Bünz, S. Agrawal, M. Zamani, D. Boneh, Zether: Towards privacy in a smart contract world, in: International Conference on Financial Cryptography and Data Security, Springer, 2020, pp. 423–443.

[18] M. F. Esgin, R. K. Zhao, R. Steinfeld, J. K. Liu, D. Liu, Matrict: efficient, scalable and post-quantum blockchain confidential transactions protocol, in: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019, pp. 567–584.

[19] M. F. Esgin, R. Steinfeld, R. K. Zhao, Matrict+: More efficient post-quantum private blockchain payments, in: 2022 IEEE Symposium on Security and Privacy (SP), IEEE, 2022, pp. 1281–1298.

[20] S.-F. Sun, M. H. Au, J. K. Liu, T. H. Yuen, Ringct 2.0: A compact accumulator-based (linkable ring signature) protocol for blockchain cryptocurrency monero, in: Computer Security–ESORICS 2017: 22nd European Symposium on Research in Computer Security, Oslo, Norway, September 11-15, 2017, Proceedings, Part II 22, Springer, 2017, pp. 456–474.

[21] T. H. Yuen, S.-f. Sun, J. K. Liu, M. H. Au, M. F. Esgin, Q. Zhang, D. Gu, Ringct 3.0 for blockchain confidential transaction: Shorter size and stronger security, in: Financial Cryptography and Data Security: 24th International Conference, FC 2020, Kota Kinabalu, Malaysia, February 10–14, 2020 Revised Selected Papers 24, Springer, 2020, pp. 464–483.

[22] Y. Li, J. Weng, M. Li, W. Wu, J. Weng, J.-N. Liu, S. Hu, Zerocross: A sidechain-based privacy-preserving cross-chain solution for monero, Journal of Parallel and Distributed Computing 169 (2022) 301–316.

[23] D. Benarroch, M. Campanelli, D. Fiore, K. Gurkan, D. Kolonelos, Zero-knowledge proofs for set membership: efficient, succinct, modular, in: International Conference on Financial Cryptography and Data Security, Springer, 2021, pp. 393–414.

[24] C. Baum, B. David, T. K. Frederiksen, P2dex: privacy-preserving decentralized cryptocurrency exchange, in: International Conference on Applied Cryptography and Network Security, Springer, 2021, pp. 163–194.

[25] C. Baum, B. David, R. Dowsley, Insured mpc: Efficient secure computation with financial penalties, in: International Conference on Financial Cryptography and Data Security, Springer, 2020, pp. 404–420.

[26] I. Bentov, Y. Ji, F. Zhang, L. Breidenbach, P. Daian, A. Juels, Tesseract: Real-time cryptocurrency exchange using trusted hardware, in: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019, pp. 1521–1538.

[27] J. Lind, O. Naor, I. Eyal, F. Kelbert, E. G. Sirer, P. Pietzuch, Teechain: a secure payment network with asynchronous blockchain access, in: Proceedings of the 27th ACM Symposium on Operating Systems Principles, 2019, pp. 63–79.

[28] J. Van Bulck, D. Oswald, E. Marin, A. Aldoseri, F. D. Garcia, F. Piessens, A tale of two worlds: Assessing the vulnerability of enclave shielding runtimes, in: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019, pp. 1741–1758.

[29] G. Chen, S. Chen, Y. Xiao, Y. Zhang, Z. Lin, T. H. Lai, Sgxpectre: Stealing intel secrets from sgx enclaves via speculative execution, in: 2019 IEEE European Symposium on Security and Privacy (EuroS&P), IEEE, 2019, pp. 142–157.

[30] N. Narula, W. Vasquez, M. Virza, zkLedger: Privacy-Preserving auditing for distributed ledgers, in: 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18), USENIX Association, Renton, WA, 2018, pp. 65–80.
URL https://www.usenix.org/conference/nsdi18/presentation/narula

[31] H. Kang, T. Dai, N. Jean-Louis, S. Tao, X. Gu, Fabzk: Supporting privacy-preserving, auditable smart contracts in hyperledger fabric, in: 2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), 2019, pp. 543–555. doi:10.1109/DSN.2019.00061.

[32] D. Rathee, G. V. Policharla, T. Xie, R. Cottone, D. Song, Zebra: Anonymous credentials with practical on-chain verification and applications to kyc in defi, Cryptology ePrint Archive, Paper 2022/1286, https://eprint.iacr.org/2022/1286 (2022).
URL https://eprint.iacr.org/2022/1286

[33] G. Jeong, N. Lee, J. Kim, H. Oh, Azeroth: Auditable zero-knowledge transactions in smart contracts, Cryptology ePrint Archive, Paper 2022/211, https://eprint.iacr.org/2022/211 (2022).
URL https://eprint.iacr.org/2022/211

[34] C. Lin, X. Huang, J. Ning, D. He, Aca: Anonymous, confidential and auditable transaction systems for blockchain, IEEE Transactions on Dependable and Secure Computing (2022).

[35] V. Buterin, J. Illum, M. Nadler, F. Schär, A. Soleimani, Blockchain privacy and regulatory compliance: Towards a practical equilibrium, Blockchain: Research and Applications 5 (1) (2024) 100176.

[36] R. W. Lai, V. Ronge, T. Ruffing, D. Schröder, S. A. K. Thyagarajan, J. Wang, Omniring: Scaling private payments without trusted setup, in: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019, pp. 31–48.

[37] K. Gjøsteen, M. Raikwar, S. Wu, Pribank: confidential blockchain scaling using short commit-and-proof nizk argument, in: Cryptographers' Track at the RSA Conference, Springer, 2022, pp. 589–619.

[38] T. Kerber, A. Kiayias, M. Kohlweiss, Kachina–foundations of private smart contracts, in: 2021 IEEE 34th Computer Security Foundations Symposium (CSF), IEEE, 2021, pp. 1–16.

[39] A. Poelstra, A. Back, M. Friedenbach, G. Maxwell, P. Wuille, Confidential assets, in: International Conference on Financial Cryptography and Data Security, Springer, 2018, pp. 43–63.

[40] Binance, Binance smart chain: A parallel binance chain to enable smart contracts, https://dex-bin.bnbstatic.com/static/Whitepaper_BinanceSmartChain.pdf (2020).
URL https://dex-bin.bnbstatic.com/static/Whitepaper_BinanceSmartChain.pdf

[41] J. Kanani, S. Nailwal, A. Arjun, Polygon lightpaper, https://polygon.technology/lightpaper-polygon.pdf (2021).
URL https://polygon.technology/lightpaper-polygon.pdf

[42] M. Raikwar, S. Wu, K. Gjøsteen, Security model for privacy-preserving blockchain-based cryptocurrency systems, in: International Conference on Network and System Security, Springer, 2023, pp. 137–152.

[43] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, M. Schofnegger, Poseidon: A new hash function for Zero-Knowledge proof systems, in: 30th USENIX Security Symposium (USENIX Security 21), USENIX Association, 2021, pp. 519–535.
URL https://www.usenix.org/conference/usenixsecurity21/presentation/grassi

[44] G. Bertoni, J. Daemen, M. Peeters, G. Van Assche, Keccak, in: T. Johansson, P. Q. Nguyen (Eds.), Advances in Cryptology – EUROCRYPT 2013, Springer Berlin Heidelberg, Berlin, Heidelberg, 2013, pp. 313–314.

[45] J. Eberhardt, S. Tai, Zokrates - scalable privacy-preserving off-chain computations, in: 2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData), 2018, pp. 1084–1091. doi:10.1109/Cybermatics_2018.2018.00199.

[46] S. Chaliasos, I. Reif, A. Torralba-Agell, J. Ernstberger, A. Kattis, B. Livshits, Analyzing and benchmarking zk-rollups, in: 6th Conference on Advances in Financial Technologies (AFT 2024), Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024, pp. 6–1.

[47] B. A. Asare, P. Branco, I. Kiringa, T. Yeap, Addshare+: Efficient selective additive secret sharing approach for private federated learning, in: 2024 IEEE 11th International Conference on Data Science and Advanced Analytics (DSAA), IEEE, 2024, pp. 1–10.